



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра математического обеспечения и стандартизации информационных
технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
по дисциплине
«Разработка клиент-серверных мобильных приложений»

Выполнил студент группы ИКБО-66-23

Смирнов А.Ю.

Принял

Дворникова Е.М.

Москва 2026

Введение

В современных реалиях разработки под платформу Android создание конкурентоспособного программного продукта невозможно без глубокого понимания принципов управления потоками данных и жизненным циклом компонентов. Использование реактивного подхода, внедрение декларативного интерфейса через Jetpack Compose и интеграция с системными API требуют от разработчика навыков проектирования сложных, отказоустойчивых систем.

Актуальность данной работы обусловлена необходимостью решения типичных проблем мобильной разработки: блокировка пользовательского интерфейса при выполнении тяжелых вычислений, нерациональное использование ресурсов аккумулятора при работе в фоне и сложность обработки асинхронных событий. В ходе выполнения практических заданий №1-14 исследуется стек технологий на языке Kotlin, включая сопрограммы (Coroutines), механизмы реактивных потоков (Flow) и архитектурные решения для взаимодействия с ОС Android на низком уровне (сенсоры, широковещательные сообщения, менеджеры задач).

Цели и задачи

Общая цель цикла практических работ:

Цель: Изучение теоретических основ и приобретение практических навыков разработки клиент-серверных приложений, использующих расширенные возможности платформы Android для работы с многозадачностью, фоновыми процессами и аппаратными датчиками.

Задачи по работам:

1. Исследование корутин (Coroutines): Изучить механизмы структурированного параллелизма, принципы работы диспетчеров (Main, IO, Default) и методы обработки исключений в иерархических задачах.
2. Проектирование фоновых служб (Services): Освоить реализацию Foreground-сервисов для задач высокой приоритетности и Bound-сервисов для организации межкомпонентного взаимодействия.
3. Автоматизация задач (WorkManager): Научиться проектировать цепочки зависимых задач, выполнять условия по ограничениям (наличие сети, заряд батареи) и гарантировать выполнение кода после перезагрузки ОС.
4. Управление системными событиями: Реализовать прием и обработку широковещательных сообщений (Broadcast) для синхронизации состояния приложения с состоянием операционной системы.
5. Планирование операций (AlarmManager): Изучить методы постановки точных будильников и механизмы восстановления расписания после системных прерываний.
6. Реактивное программирование (Kotlin Flow): Внедрить современные методы обработки потоков данных (StateFlow, SharedFlow) для обеспечения бесшовного обновления UI.
7. Интеграция с аппаратным уровнем: Реализовать алгоритмы получения и фильтрации данных от датчиков ориентации и позиционирования (GPS, акселерометр, компас).

Задание 1: Параллельная загрузка и обработка нескольких источников данных

Необходимо создать консольную программу, которая делает следующее:

- Запускает три независимые «долгие» операции параллельно:
 1. Загрузка списка пользователей (delay 1800 мс) и возвращает `List<String>` имён
 2. Загрузка статистики продаж за день (delay 1200 мс) и возвращает `Map<String, Int>`
 3. Получение текущей погоды в 3 городах (delay 2500 мс) и возвращает `List<String>` строк вида "Москва: -3°C"
- Вывод результата только после того, как все три задачи завершатся;
- Измерить общее время выполнения;
- Добавить обработку случайного сбоя;
- Если хотя бы одна задача упала - вывести понятное сообщение, а не исключение всей программе (используй `try-catch` внутри `async` или `CoroutineExceptionHandler`).

Ожидаемое время выполнения примерно 2.5 сек

Теоретическая часть

Для реализации использовались:

- `runBlocking` — точка входа в корутины;
- `async` — для параллельного выполнения задач;
- `await()` — ожидание результата;
- `delay()` — имитация сетевого запроса;
- `measureTimeMillis` — измерение времени выполнения;
- `try-catch` — обработка исключений внутри `async`.

Параллельное выполнение позволяет значительно сократить общее время выполнения программы по сравнению с последовательным запуском.

```

import kotlinx.coroutines.*

import kotlin.system.measureTimeMillis

import kotlin.random.Random

/**
 * Главная функция программы, демонстрирующая параллельное
 * выполнение
 *
 * нескольких асинхронных операций с возможными сбоями
 */

fun main() = runBlocking { // runBlocking создает корутину и блокирует
    текущий поток до её завершения

    println("Запуск параллельных операций...\n")

    // measureTimeMillis измеряет время выполнения блока кода в
    миллисекундах

    val time = measureTimeMillis {

        // async создает дочерние корутины, которые выполняются
        параллельно

        // Каждая корутина возвращает результат типа Deferred (обещание)

        val usersDeferred = async {

            try {

                loadUsers() // Загружаем список пользователей
            }
        }
    }
}

```

```

    } catch (e: Exception) {

        println("Ошибка при загрузке пользователей: ${e.message}")

        null // В случае ошибки возвращаем null

    }
}

val salesDeferred = async {

    try {

        loadSales() // Загружаем данные о продажах

    } catch (e: Exception) {

        println("Ошибка при загрузке продаж: ${e.message}")

        null

    }

}

val weatherDeferred = async {

    try {

        loadWeather() // Загружаем данные о погоде

    } catch (e: Exception) {

        println("Ошибка при загрузке погоды: ${e.message}")

        null

    }

}

```

```

// await() приостанавливает выполнение текущей корутины
// до получения результата от соответствующей Deferred задачи
val users = usersDeferred.await()
val sales = salesDeferred.await()
val weather = weatherDeferred.await()

// Проверяем, все ли данные успешно загружены (не равны null)
if (users != null && sales != null && weather != null) {
    println("Все данные успешно получены:\n")

    println("Пользователи:")
    users.forEach { println(it) } // Выводим каждого пользователя

    println("\nПродажи:")
    sales.forEach { (product, revenue) -> // Деструктуризация пары ключ-
значение
        println("$product -> $revenue руб.")
    }

    println("\nПогода:")
    weather.forEach { println(it) } // Выводим данные о погоде
} else {
    println("\nОдна или несколько операций завершились с ошибкой.")
}

```

```

    }

}

println("\nОбщее время выполнения: ${time} мс") // Выводим общее
время выполнения
}

/**
 * Загружает список пользователей с имитацией задержки
 * @return список имен пользователей
 */
suspend fun loadUsers(): List<String> {
    delay(1800) // Имитация задержки сети (1.8 секунды)
    randomFailure() // Возможный случайный сбой

    return listOf("Alice", "Bob", "Ivan", "Olga")
}

/**
 * Загружает данные о продажах с имитацией задержки
 * @return карту товаров и их стоимости
 */
suspend fun loadSales(): Map<String, Int> {
    delay(1200) // Имитация задержки сети (1.2 секунды)

```



```

randomFailure() // Возможный случайный сбой

return mapOf(
    "Coffee" to 1680,
    "Tea" to 475
)
}

/**
 * Загружает данные о погоде с имитацией задержки
 * @return список погодных данных для разных городов
 */
suspend fun loadWeather(): List<String> {
    delay(2500) // Имитация задержки сети (2.5 секунды)
    randomFailure() // Возможный случайный сбой

    return listOf(
        "Москва: -18°C",
        "New York: -5°C",
        "Tokyo: 11°C"
    )
}

```

```

/**
 * Имитирует случайный сбой соединения
 * @throws Exception в 20% случаев (когда выпадает число 1)
 */
fun randomFailure() {
    // Random.nextInt(0, 5) генерирует число от 0 до 4
    // Вероятность сбоя = 1/5 = 20%
    if (Random.nextInt(0, 5) == 1) {
        throw Exception("Случайный сбой соединения")
    }
}

```

<pre> "C:\Program Files\Java\jdk-21\bin\java.exe" "-jav === Результаты === Пользователи: Alice Bob Ivan Olga Продажи: Coffee -> 42 Tea -> 19 Погода: Москва: -18°C New York: -5°C Токуо: 11°C Общее время выполнения: 2540 ms Process finished with exit code 0 </pre>	<pre> "C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C Ошибка загрузки продаж: Случайный сбой в Sales service === Результаты === Пользователи: Alice Bob Ivan Olga Продажи: Погода: Москва: -18°C New York: -5°C Токуо: 11°C Общее время выполнения: 2535 ms Process finished with exit code 0 </pre>
--	---

Задание 2: Поиск дубликатов в больших файлах с timeout и отменой

Необходимо создать консольную программу, которая делает следующее:

- Принимаем путь к директории (можно захардкодить или через аргументы);
- Находим все json-файлы в директории и поддиректориях;
- Для каждого файла вычисляем SHA-256 содержимого (suspend функция);
- Запускаем поиск параллельно;
- Ищем файлы-дубликаты по SHA-256;
- У всей операции есть общий таймаут n секунд (с withTimeoutOrNull);
- Если время вышло - отменяем все корутины и выводим «Поиск прерван по таймауту»;
- Выводим группы дубликатов (если нашлись).

Для вычисления SHA-256 можно использовать: **java.security.MessageDigest**

```
import kotlinx.coroutines.*

import java.io.File
import java.nio.file.Files
import java.nio.file.Paths
import java.security.MessageDigest
import kotlin.system.measureTimeMillis

fun main() = runBlocking { // Создание корутины

    val directoryPath = "C:/test_data" // Путь к директории
```

```
val timeoutSeconds = 5L // Таймаут выполнения

println("Начало поиска дубликатов...\n")

val time = measureTimeMillis {

    // Выполнение с ограничением по времени
    val result = withTimeoutOrNull(timeoutSeconds * 1000) {

        coroutineScope { // Область видимости для корутин

            // Поиск всех JSON файлов
            val jsonFiles = Files.walk(Paths.get(directoryPath))
                .filter { Files.isRegularFile(it) && it.toString().endsWith(".json") }
                .map { itToFile() }
                .toList()

            println("Найдено JSON файлов: ${jsonFiles.size}")

            // Параллельное вычисление хэшей
            val deferredList = jsonFiles.map { file ->
                async(Dispatchers.IO) { // Асинхронная задача в IO потоке
                    val hash = calculateSHA256(file)
                }
            }
        }
    }
}
```

```

        file.absolutePath to hash // Путь и хэш файла
    }
}

val results = deferredList.awaitAll() // Ожидание всех задач

// Группировка дубликатов по хэшу
results.groupBy({ it.second }, { it.first })

    .filter { it.value.size > 1 } // Только файлы с одинаковым хэшем
}
}

// Вывод результатов
if (result == null) {
    println("\nПоиск прерван по таймауту.")
} else {
    if (result.isEmpty()) {
        println("\nДубликаты не найдены.")
    } else {
        println("\nНайдены группы дубликатов:\n")
        result.forEach { (hash, files) ->
            println("Хэш: $hash")
            files.forEach { println(" $it") }
        }
    }
}

```

```

        println()

    }

}

}

}

println("Общее время выполнения: ${time} мс")
}

/**
 * Вычисление SHA-256 хэша файла
 */
suspend fun calculateSHA256(file: File): String = withContext(Dispatchers.IO) {
    val digest = MessageDigest.getInstance("SHA-256")

    val bytes = file.readBytes() // Чтение файла

    val hashBytes = digest.digest(bytes)

    // Преобразование в HEX строку

    hashBytes.joinToString("") { "%02x".format(it) }
}

```

```

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\Jet
Найдено файлов: 3

Группы дубликатов:

SHA256: 3cba1e3cf23c8ce24b7e08171d823fbd9a4929aafd9f27516e30699d3a42026a
C:\Users\egorl\Documents\Task1\src\main\resources\test\a.json
C:\Users\egorl\Documents\Task1\src\main\resources\test\b.json

Process finished with exit code 0

```

```

"C:\Program Files\Java\jdk-21\bin\jav
Найдено файлов: 0

Группы дубликатов:
Дубликаты не найдены

Process finished with exit code 0

```


Задание 3: Экран поиска репозитория GitHub с debounce

Debounce — это когда мы говорим функции: «Не выполняйся сразу. Подожди, пока меня перестанут дёргать хотя бы N миллисекунд. Тогда выполнись один раз с самыми свежими данными». Дополнительно можно почитать тут:

<https://kotlin.coroutines.ru/2025/12/25/%D1%83%D0%BF%D1%80%D0%B0%D0%B2%D0%BB%D0%B5%D0%BD%D0%B8%D0%B5-%D0%BF%D0%BE%D1%82%D0%BE%D0%BA%D0%BE%D0%BC-%D0%B2-kotlin-flow/>

Самая простая реализация выглядит так:

```
import kotlinx.coroutines.*

fun <T> CoroutineScope.debounce(
    waitMs: Long = 500L, destinationFunction: (T) -> Unit): (T) -> Unit {

    var debounceJob: Job? = null

    return { param: T ->

        debounceJob?.cancel()

        debounceJob = launch {

            delay(waitMs)

            destinationFunction(param)

        }

    }

}
```

Необходимо создать Android приложение.

Требования к UI:

- есть поле ввода;
- при вводе текста происходит «поиск» репозитория;
- результаты отображаются в LazyColumn;

- поиск запускается не на каждый символ, а с задержкой (debounce);
- во время поиска показывается индикатор загрузки;
- предыдущий поиск автоматически отменяется, если пользователь продолжает печатать;

Требования к корутинам:

- Использовать launch/async/withContext/delay;
- Реализовать debounce (можно с помощью delay + cancel);
- Отмена предыдущей задачи при новом вводе текста (через Job.cancel())
- Всё должно работать внутри rememberCoroutineScope()

Источник данных - большой json-файл github_repos.json в assets или res/raw (можно взять ~50–200 записей репозитория с полями: id, full_name, description, stargazers_count, language)

```
import android.os.Bundle

import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import kotlinx.coroutines.*
import org.json.JSONArray
```

```
import java.io.BufferedReader

import java.io.InputStreamReader

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContent {

            SearchScreen()

        }

    }

}

@Composable

fun SearchScreen() {

    val scope = rememberCoroutineScope() // Для запуска корутин в Compose

    // Состояния UI

    var query by remember { mutableStateOf("") } // Текст поиска

    var results by remember { mutableStateOf<List<GithubRepo>>(emptyList()) } // Результаты

    var isLoading by remember { mutableStateOf(false) } // Флаг загрузки
```

```

// Ссылка на текущую задачу поиска для возможности отмены
var searchJob by remember { mutableStateOf<Job?>(null) }

Column(modifier = Modifier
    .fillMaxSize()
    .padding(16.dp)) {

    OutlinedTextField(
        value = query,
        onChange = { text ->

            query = text

            searchJob?.cancel() // Отмена предыдущего поиска

            // Запуск нового поиска с debounce
            searchJob = scope.launch {

                delay(500) // Задержка 500мс перед поиском

                if (query.isNotBlank()) {
                    isLoading = true
                }
            }
        }
    )
}

```

```

        // Загрузка и фильтрация в фоновом потоке

        val data = withContext(Dispatchers.IO) {

            loadReposFromAssets().filter {

                it.full_name.contains(query, ignoreCase = true)

            }

        }

        results = data

        isLoading = false

    } else {

        results = emptyList()

    }

},

label = { Text("Поиск репозитория") },

modifier = Modifier.fillMaxWidth()

)

Spacer(modifier = Modifier.height(16.dp))

if (isLoading) {

    CircularProgressIndicator() // Индикатор загрузки

}

```

```

        Spacer(modifier = Modifier.height(16.dp))

        // Список результатов
        LazyColumn {
            items(results) { repo ->
                RepoItem(repo)
            }
        }
    }
}

@Composable
fun RepoItem(repo: GithubRepo) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = 8.dp)
    ) {
        Column(modifier = Modifier.padding(12.dp)) {
            Text(text = repo.full_name, style =
MaterialTheme.typography.titleMedium)

            Text(text = repo.description ?: "") // Описание может быть null

            Text(text = "★ ${repo.stargazers_count} | ${repo.language}")
        }
    }
}

```

```

    }

    }

}

fun loadReposFromAssets(): List<GithubRepo> {

    val context = androidx.compose.ui.platform.LocalContext.current

    val inputStream = context.assets.open("github_repos.json")

    val reader = BufferedReader(InputStreamReader(inputStream))

    val jsonText = reader.readText()

    // Парсинг JSON

    val jsonArray = JSONArray(jsonText)

    val list = mutableListOf<GithubRepo>()

    for (i in 0 until jsonArray.length()) {

        val obj = jsonArray.getJSONObject(i)

        list.add(

            GithubRepo(

                id = obj.getInt("id"),

                full_name = obj.getString("full_name"),

                description = obj.optString("description"), // optString возвращает ""
если null

                stargazers_count = obj.getInt("stargazers_count"),

```

```
        language = obj.optString("language")

    )

)

}

return list
}

// Класс данных для репозитория
data class GithubRepo(

    val id: Int,

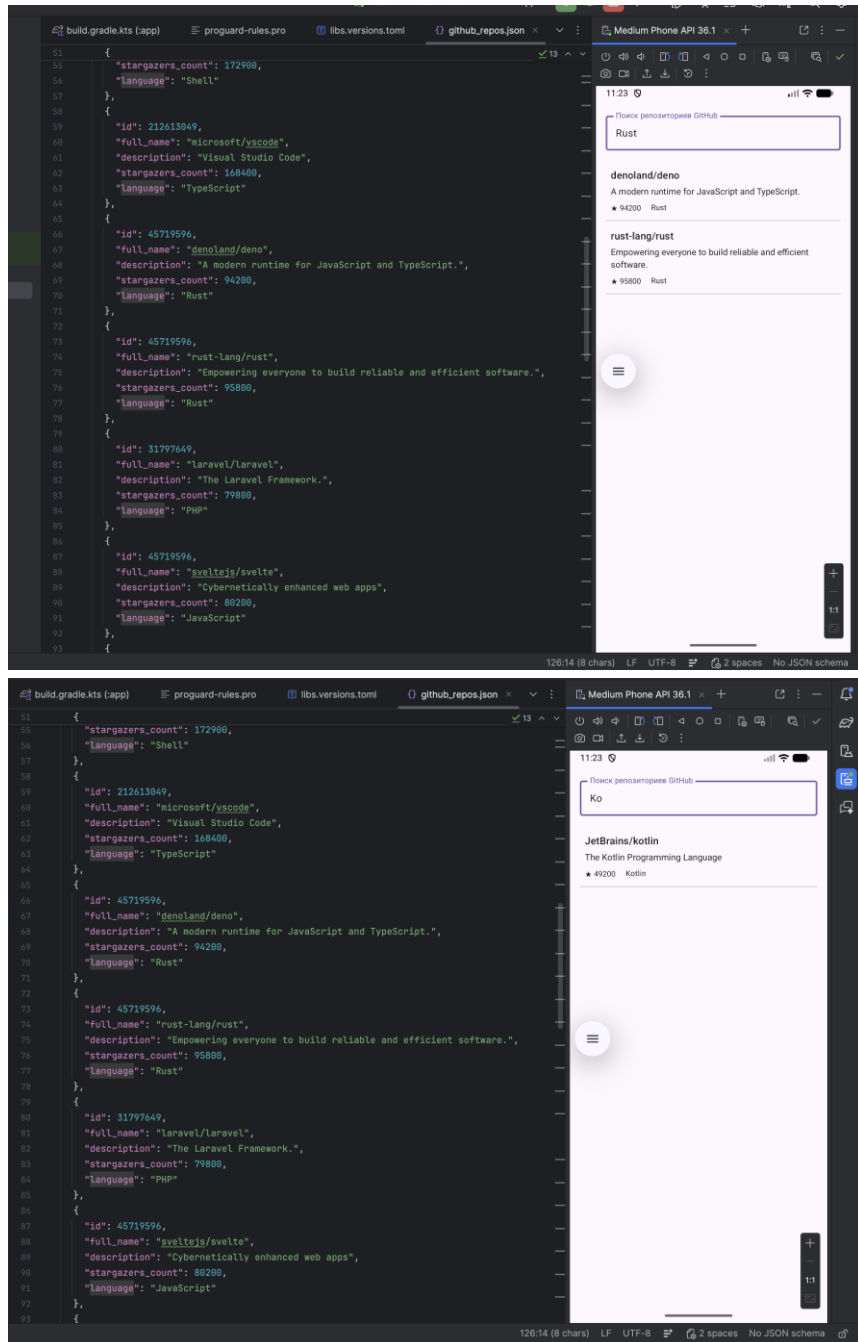
    val full_name: String,

    val description: String?,

    val stargazers_count: Int,

    val language: String?

)
```



Задание 4: Социальная лента с постами, аватарками и комментариями

Необходимо создать Android приложение. Требования к UI:

- при запуске экрана загружаются 8–15 постов из JSON;
- для каждого поста параллельно загружаются:
 - аватарка автора (имитация: delay + строка-цвет или url);
 - список комментариев (delay + фильтр по postId);
- посты отображаются по мере готовности (не ждём всех сразу);
- каждая карточка поста показывает своё состояние загрузки (Loading / Ready / Error);
- есть кнопка «Обновить» — при нажатии отменяются все текущие загрузки аватарок и комментариев;
- если комментарии или аватарка не загрузились - показывается placeholder / иконка ошибки;

Требования к корутинам:

- rememberCoroutineScope() для запуска задач из composable;
- coroutineScope { ... } или supervisorScope { ... } для параллельной загрузки подзадач одного поста;
- async + await() / awaitAll() для получения результатов;
- Job.cancel() для отмены всех дочерних задач при обновлении;
- withContext(Dispatchers.IO) для чтения JSON;
- искусственные delay() для имитации сети;
- обработка исключений (try-catch внутри async);

Источник данных - локальные json-файлы в assets или res/raw

```
import android.os.Bundle

import androidx.activity.ComponentActivity

import androidx.activity.compose.setContent
```

```

import androidx.compose.foundation.layout.*

import androidx.compose.foundation.lazy.LazyColumn

import androidx.compose.foundation.lazy.items

import androidx.compose.material3.*

import androidx.compose.runtime.*

import androidx.compose.ui.Modifier

import androidx.compose.ui.unit.dp

import kotlinx.coroutines.*

import org.json.JSONArray

import java.io.BufferedReader

import java.io.InputStreamReader

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContent { SocialScreen() }

    }

}

// Состояния загрузки для каждого поста

sealed class PostState {

    object Loading : PostState()

    data class Error(val message: String) : PostState()

```

```

data class Ready(val avatar: String?, val comments: List<Comment>) :
PostState()

}

@Composable
fun SocialScreen() {

    val scope = rememberCoroutineScope()

    // Состояния UI
    var posts by remember { mutableStateOf<List<Post>>(emptyList()) }
    var postStates by remember { mutableStateOf<Map<Int,
PostState>>(emptyMap()) }

    var parentJob by remember { mutableStateOf<Job?>(null) }

    // Функция загрузки ленты
    fun loadFeed() {
        parentJob?.cancel() // Отмена предыдущей загрузки

        parentJob = scope.launch {
            // Загрузка списка постов
            posts = loadPosts()

            // Устанавливаем начальное состояние Loading для всех постов

```

```

postStates = posts.associate { it.id to PostState.Loading }

// Для каждого поста запускаем отдельную корутину
posts.forEach { post ->

    launch {

        // supervisorScope - ошибка в одном посте не отменяет другие
        val state = supervisorScope {

            try {

                // Параллельная загрузка аватара и комментариев

                val avatarDeferred = async {

                    delay(800)

                    randomFailure() // Имитация сбоя

                    post.avatarUrl

                }

                val commentsDeferred = async {

                    delay(1200)

                    randomFailure()

                    loadComments().filter { it.postId == post.id }

                }

                // Ожидание обоих результатов

                PostState.Ready(

```

```

        avatarDeferred.await(),
        commentsDeferred.await()
    )
} catch (e: Exception) {
    PostState.Error("Ошибка загрузки")
}
}

// Обновляем состояние конкретного поста
postStates = postStates.toMutableMap().apply {
    put(post.id, state)
}
}
}
}

// Загрузка при первом запуске
LaunchedEffect(Unit) { loadFeed() }

Column(Modifier.fillMaxSize().padding(16.dp)) {

    Button(onClick = { loadFeed() }) {

```

```

        Text("Обновить")

    }

    Spacer(Modifier.height(16.dp))

    // Список постов
    LazyColumn {
        items(posts) { post ->
            PostCard(post, postStates[post.id] ?: PostState.Loading)
            Spacer(Modifier.height(12.dp))
        }
    }
}

@Composable
fun PostCard(post: Post, state: PostState) {

    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(12.dp)) {

            Text(post.title, style = MaterialTheme.typography.titleMedium)

            Text(post.body)

```

```
Spacer(Modifier.height(8.dp))
```

```
// Отображение разных состояний загрузки
```

```
when (state) {
```

```
    is PostState.Loading -> {
```

```
        Text("Загрузка...")
```

```
        CircularProgressIndicator()
```

```
    }
```

```
    is PostState.Error -> {
```

```
        Text("⚠️ ${state.message}")
```

```
    }
```

```
    is PostState.Ready -> {
```

```
        Text("Аватар: ${state.avatar ?: "нет"}")
```

```
        Spacer(Modifier.height(4.dp))
```

```
        Text("Комментарии:")
```

```
        state.comments.forEach {
```

```
            Text("- ${it.name}: ${it.body}")
```

```
        }
```

```
    }
```

```

    }

    }

}

}

// Загрузка постов из assets

suspend fun loadPosts(): List<Post> = withContext(Dispatchers.IO) {

    val context = androidx.compose.ui.platform.LocalContext.current

    val inputStream = context.assets.open("social_posts.json")

    val text = BufferedReader(InputStreamReader(inputStream)).readText()

    val jsonArray = JSONArray(text)

    val list = mutableListOf<Post>()

    for (i in 0 until jsonArray.length()) {

        val obj = jsonArray.getJSONObject(i)

        list.add(

            Post(

                id = obj.getInt("id"),

                userId = obj.getInt("userId"),

                title = obj.getString("title"),

                body = obj.getString("body"),

                avatarUrl = obj.getString("avatarUrl")

            )

        )

    }

}

```



```

    )
}
list
}

// Загрузка комментариев из assets
suspend fun loadComments(): List<Comment> = withContext(Dispatchers.IO) {
    val context = androidx.compose.ui.platform.LocalContext.current
    val inputStream = context.assets.open("comments.json")
    val text = BufferedReader(InputStreamReader(inputStream)).readText()
    val jsonArray = JSONArray(text)

    val list = mutableListOf<Comment>()
    for (i in 0 until jsonArray.length()) {
        val obj = jsonArray.getJSONObject(i)
        list.add(
            Comment(
                postId = obj.getInt("postId"),
                id = obj.getInt("id"),
                name = obj.getString("name"),
                body = obj.getString("body")
            )
        )
    }
}

```

```

    }

    list
}

// Имитация случайного сбоя (1/6 вероятность)
fun randomFailure() {
    if ((0..5).random() == 1) {
        throw Exception("Network error")
    }
}

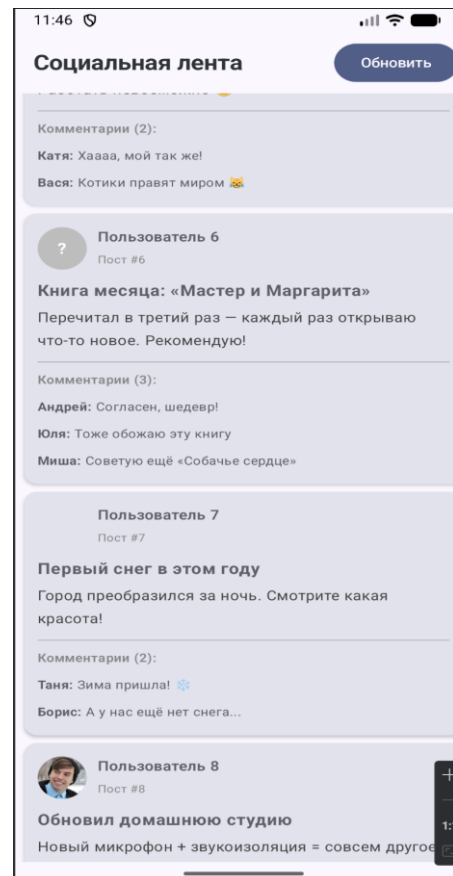
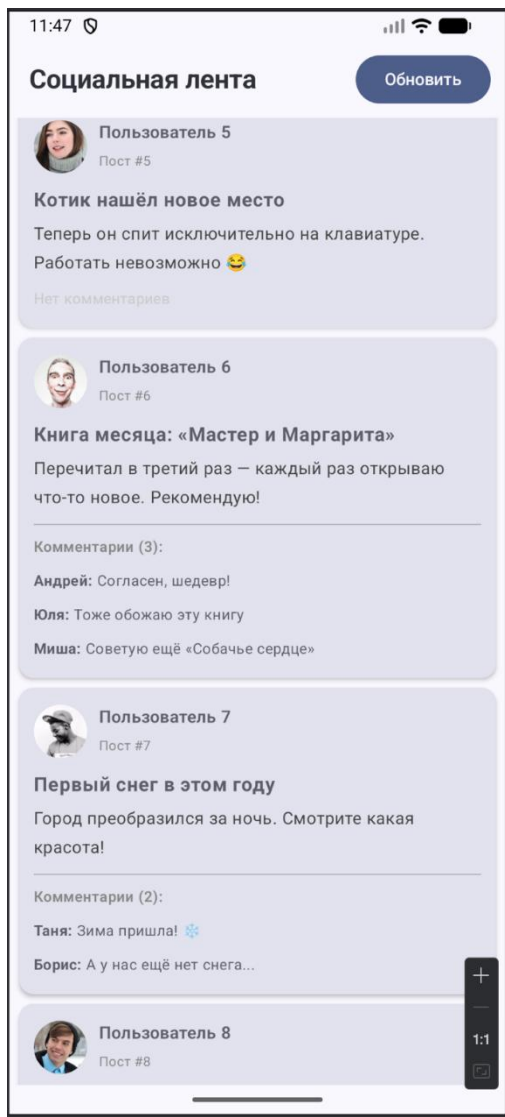
data class Post(
    val id: Int,
    val userId: Int,
    val title: String,
    val body: String,
    val avatarUrl: String
)

data class Comment(
    val postId: Int,
    val id: Int,
    val name: String,

```

val body: String

)



Задание 5 (Foreground service): Счётчик времени с уведомлением

Функционал приложения:

- При нажатии «Старт» запускается foreground-сервис;
- Сервис каждую секунду увеличивает счётчик (секунды);
- В уведомлении отображается: «Прошло X секунд»;
- Приложение показывает текущее значение крупным шрифтом;
- Кнопка «Стоп» останавливает сервис.

```
package com.example.timerapp

import android.app.*
import android.content.Intent
import android.os.IBinder
import androidx.core.app.NotificationCompat
import kotlinx.coroutines.*

class TimerService : Service() {

    private var seconds = 0
    private var serviceJob: Job? = null

    companion object {
        const val CHANNEL_ID = "timer_channel"
        const val ACTION_UPDATE = "ACTION_UPDATE"
        const val EXTRA_SECONDS = "EXTRA_SECONDS"
    }

    override fun onCreate() {
        super.onCreate()
        createNotificationChannel()
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {

        startForeground(1, createNotification("Прошло 0 секунд"))

        serviceJob = CoroutineScope(Dispatchers.Default).launch {
            while (isActive) {
                delay(1000)
                seconds++

                updateNotification(seconds)
                sendUpdate(seconds)
            }
        }

        return START_STICKY
    }

    private fun updateNotification(sec: Int) {
        val notification = createNotification("Прошло $sec секунд")
        val manager = getSystemService(NotificationManager::class.java)
        manager.notify(1, notification)
    }
}
```

```

    }

    private fun sendUpdate(sec: Int) {
        val intent = Intent(ACTION_UPDATE)
        intent.putExtra(EXTRA_SECONDS, sec)
        sendBroadcast(intent)
    }

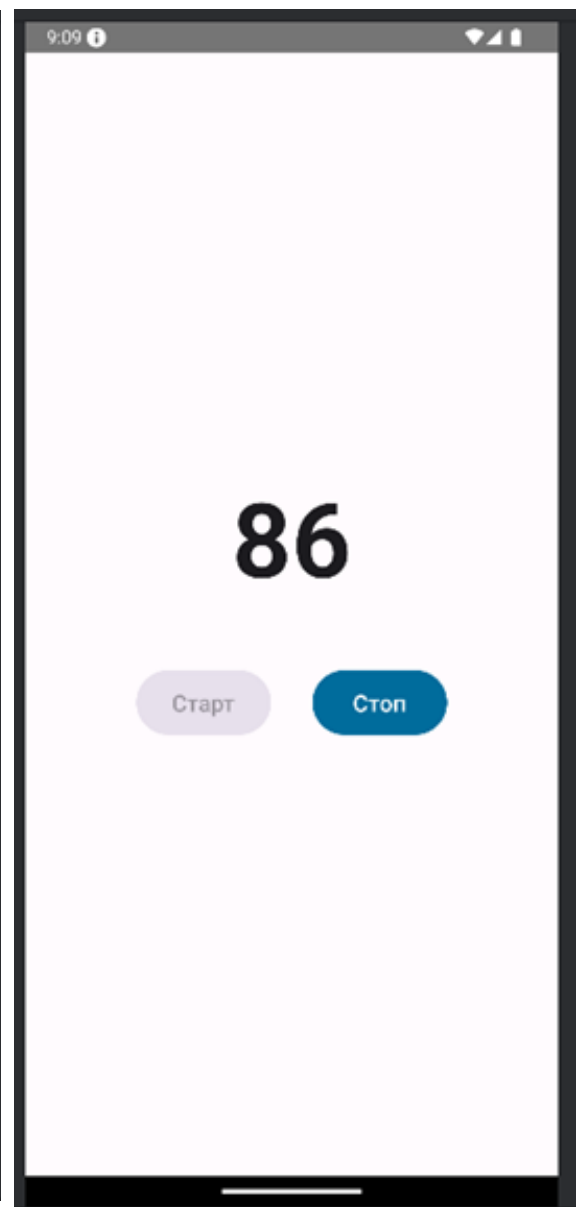
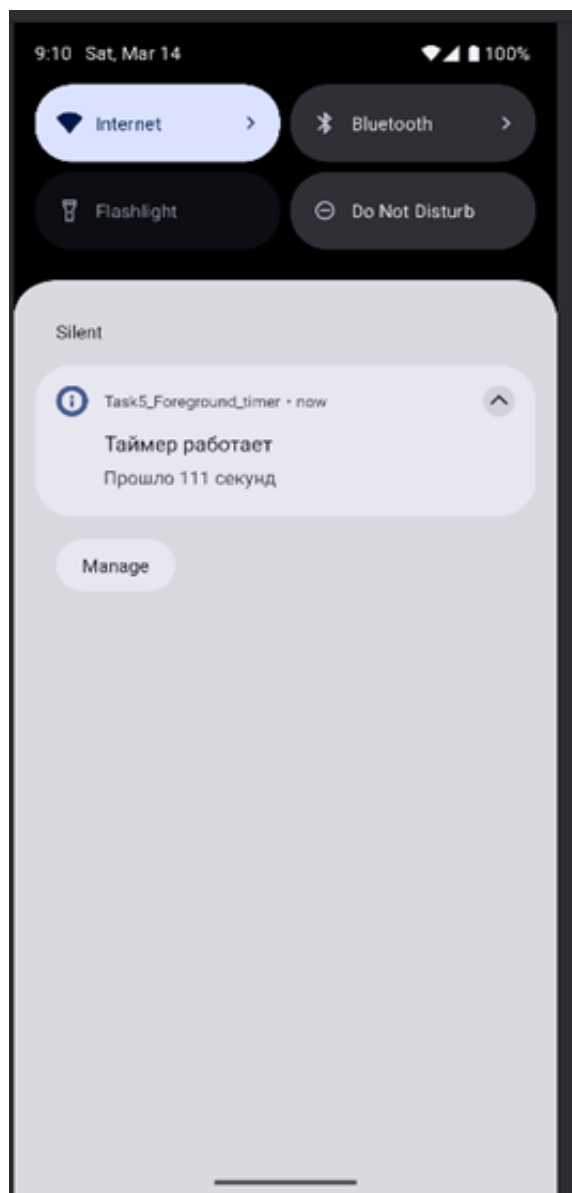
    private fun createNotification(text: String): Notification {
        return NotificationCompat.Builder(this, CHANNEL_ID)
            .setContentTitle("Таймер работает")
            .setContentText(text)
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setOngoing(true)
            .build()
    }

    private fun createNotificationChannel() {
        val channel = NotificationChannel(
            CHANNEL_ID,
            "Timer Channel",
            NotificationManager.IMPORTANCE_LOW
        )
        val manager = getSystemService(NotificationManager::class.java)
        manager.createNotificationChannel(channel)
    }

    override fun onDestroy() {
        serviceJob?.cancel()
        super.onDestroy()
    }

    override fun onBind(intent: Intent?): IBinder? = null
}

```



Задание 6 (Background service): Одноразовый таймер с уведомлением

Функционал приложения:

- Пользователь вводит количество секунд (например: 30);
- Нажимает «Запустить таймер»;
- Сервис ждёт указанное время и показывает уведомление «Таймер завершён!» с сообщением «Время вышло»;
- Сервис сам себя останавливает.

```
package com.example.backgroundtimer

import android.app.*
import android.content.Intent
import android.os.IBinder
import androidx.core.app.NotificationCompat
import kotlin.coroutines.*

class TimerService : Service() {

    private var serviceJob: Job? = null // Для управления корутиной

    companion object {
        const val EXTRA_SECONDS = "EXTRA_SECONDS"
        const val CHANNEL_ID = "timer_done_channel"
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {

        // Получаем количество секунд из Intent
        val seconds = intent?.getIntExtra(EXTRA_SECONDS, 0) ?: 0

        // Запускаем корутину для отсчета времени
        serviceJob = CoroutineScope(Dispatchers.Default).launch {

            delay(seconds * 1000L) // Ожидание указанного времени

            showNotification() // Показываем уведомление о завершении
        }
    }
}
```

```

        stopSelf() // Останавливаем сервис
    }

    return START_NOT_STICKY // Сервис НЕ будет перезапущен после убийства
}

private fun showNotification() {

    val manager = getSystemService(NotificationManager::class.java)

    // Создаем канал с высоким приоритетом (будет звук/вибрация)
    val channel = NotificationChannel(
        CHANNEL_ID,
        "Timer Finished",
        NotificationManager.IMPORTANCE_HIGH // Высокий приоритет
    )
    manager.createNotificationChannel(channel)

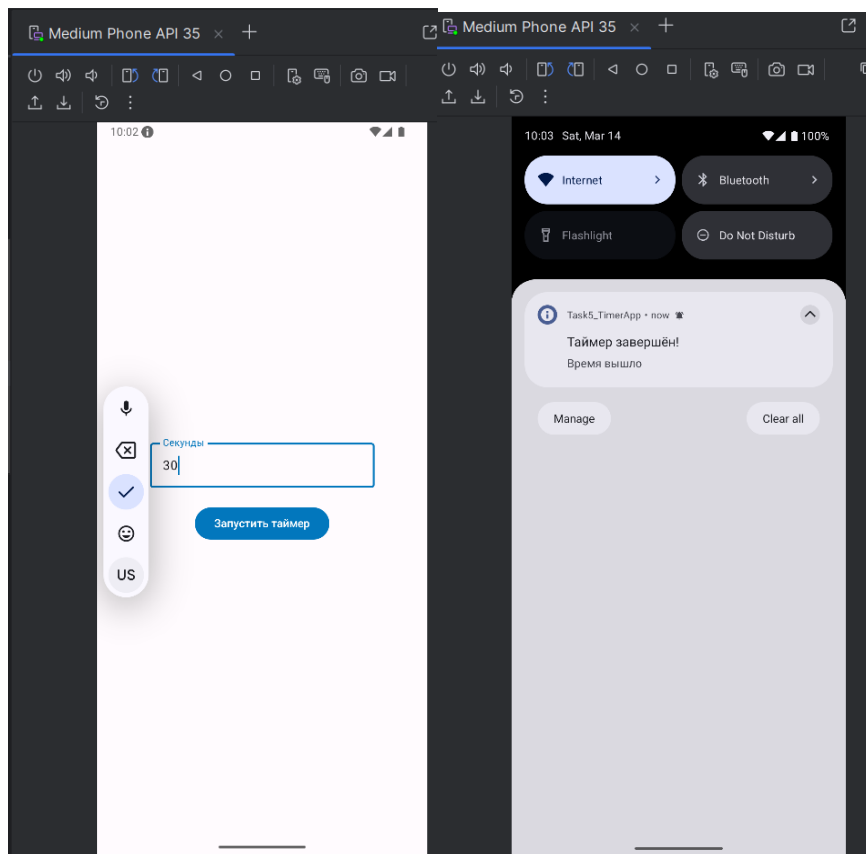
    // Создаем уведомление
    val notification = NotificationCompat.Builder(this, CHANNEL_ID)
        .setContentTitle("Таймер завершён!")
        .setContentText("Время вышло")
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setAutoCancel(true) // Уведомление исчезнет при нажатии
        .build()

    manager.notify(2, notification) // Показываем уведомление с ID=2
}

override fun onDestroy() {
    serviceJob?.cancel() // Отмена корутины при остановке
    super.onDestroy()
}

override fun onBind(intent: Intent?): IBinder? = null
}

```

Задание 7 (Bind service): Случайное число каждую секунду

Функционал приложения:

- Приложение подключается к сервису;
- Сервис каждую секунду генерирует случайное число 0 ..100;
- UI показывает последнее число;
- Кнопки «Подключиться» / «Отключиться».

Реализация сервиса

RandomNumberService.kt

```
package com.example.bindservice

import android.app.Service
import android.content.Intent
import android.os.Binder
import android.os.IBinder
import kotlinx.coroutines.*
import kotlin.random.Random

class RandomNumberService : Service() {

    // Binder для связи с Activity
    private val binder = LocalBinder()
    private var serviceJob: Job? = null

    private var currentNumber = 0 // Текущее сгенерированное число

    // Внутренний класс Binder для получения ссылки на сервис
    inner class LocalBinder : Binder() {
        fun getService(): RandomNumberService = this@RandomNumberService
    }
}
```

```

}

// Вызывается при привязке Activity к сервису
override fun onBind(intent: Intent?): IBinder {
    startGenerating() // Запускаем генерацию чисел
    return binder // Возвращаем Binder для связи
}

// Вызывается при отвязке всех клиентов
override fun onUnbind(intent: Intent?): Boolean {
    stopGenerating() // Останавливаем генерацию
    return super.onUnbind(intent)
}

// Публичный метод для получения текущего числа из Activity
fun getCurrentNumber(): Int = currentNumber

// Запуск генерации случайных чисел
private fun startGenerating() {
    serviceJob = CoroutineScope(Dispatchers.Default).launch {
        while (isActive) {
            delay(1000) // Каждую секунду
            currentNumber = Random.nextInt(0, 101) // Число от 0 до 100
        }
    }
}

// Остановка генерации
private fun stopGenerating() {

```

```
        serviceJob?.cancel()
    }
}
```

Реализация MainActivity (Jetpack Compose)

MainActivity.kt

```
package com.example.bindservice

import android.content.*
import android.os.Bundle
import android.os.IBinder
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import kotlinx.coroutines.*

class MainActivity : ComponentActivity() {

    // Ссылка на сервис и флаг привязки
    private var service: RandomNumberService? = null
    private var isBound = false

    // ServiceConnection для отслеживания состояния привязки
```

```

private val connection = object : ServiceConnection {

    // Вызывается когда сервис успешно запущен и привязан
    override fun onServiceConnected(name: ComponentName?, binder:
IBinder?) {
        val localBinder = binder as RandomNumberService.LocalBinder
        service = localBinder.getService() // Получаем ссылку на сервис
        isBound = true
    }

    // Вызывается при потере соединения (например, сервис упал)
    override fun onServiceDisconnected(name: ComponentName?) {
        isBound = false
        service = null
    }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    setContent {
        BindServiceScreen(
            onBind = { bindToService() },
            onUnbind = { unbindFromService() },
            serviceProvider = { service },
            isBoundProvider = { isBound }
        )
    }
}

```

```

// Привязка к сервису
private fun bindToService() {
    val intent = Intent(this, RandomNumberService::class.java)
    bindService(intent, connection, Context.BIND_AUTO_CREATE)
}

// Отвязка от сервиса
private fun unbindFromService() {
    if (isBound) {
        unbindService(connection)
        isBound = false
    }
}

// Отвязываемся при уничтожении Activity
override fun onDestroy() {
    if (isBound) {
        unbindService(connection)
    }
    super.onDestroy()
}

}

// Экран с UI на Compose
@Composable
fun BindServiceScreen(
    onBind: () -> Unit, // Функция привязки
    onUnbind: () -> Unit, // Функция отвязки

```

```

serviceProvider: () -> RandomNumberService?, // Получение сервиса
isBoundProvider: () -> Boolean // Проверка статуса привязки
) {

    var currentNumber by remember { mutableStateOf("-") }
    val scope = rememberCoroutineScope()

    // Эффект для обновления числа каждую секунду когда сервис привязан
    LaunchedEffect(isBoundProvider()) {
        if (isBoundProvider()) {
            scope.launch {
                while (true) {
                    delay(1000) // Обновление каждую секунду
                    val number = serviceProvider?.getCurrentNumber()
                    currentNumber = number?.toString() ?: "-"
                }
            }
        } else {
            currentNumber = "-" // Сброс при отвязке
        }
    }

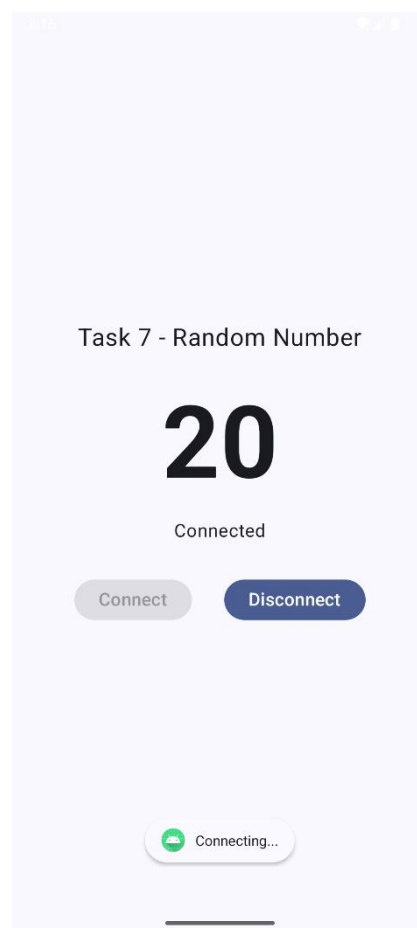
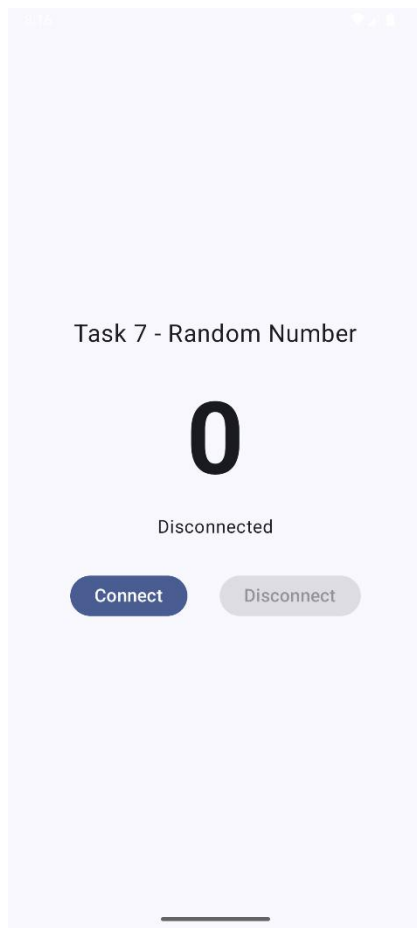
    // UI
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {

```

```
// Отображение текущего числа
Text(
    text = currentNumber,
    fontSize = 64.sp
)

Spacer(modifier = Modifier.height(32.dp))

// Кнопка меняется в зависимости от состояния
if (!isBoundProvider()) {
    Button(onClick = onBind) {
        Text("Подключиться")
    }
} else {
    Button(onClick = onUnbind) {
        Text("Отключиться")
    }
}
}
```

Задание 8 (Последовательная цепочка): Обработка фото перед загрузкой в облако

Функционал приложения:

Пользователь выбрал фото - приложение должно выполнить следующий набор действий:

1. Сжать фото (Worker 1);
2. Добавить водяной знак или подпись (Worker 2);
3. Загрузить готовое фото в облако (Worker 3, имитация).

Каждый шаг зависит от предыдущего - последовательная цепочка.

Требования к реализации:

- Одна кнопка «Начать обработку и загрузку»;
- Показывать текущий шаг крупным текстом (например: «Сжимаем фото...», «Добавляем водяной знак...», «Загружаем...»);
- Показывать прогресс-бар (имитация 0–100% для каждого шага);
- В конце — сообщение «Готово! Фото загружено» + путь/имя файла;
- Если любой шаг упал - показать ошибку и отменить всю цепочку;

Минимальный UI:

- Text (headline) — текущий статус;
- LinearProgressIndicator (когда работает);
- Button «Начать обработку» (disabled во время работы);
- Text — результат или ошибка.

Реализация Worker-классов

```
import android.content.Context
import androidx.work.CoroutineWorker
```

```

import androidx.work.WorkerParameters
import androidx.work.workDataOf
import kotlinx.coroutines.delay

/**
 * WorkManager workers для последовательной обработки изображения:
 * Сжатие -> Водяной знак -> Загрузка в облако
 */

// 1. Воркер для сжатия изображения
class CompressWorker(context: Context, params: WorkerParameters) :
    CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        try {
            // Имитация процесса сжатия с прогрессом
            for (i in 0..100 step 20) { // Шаг 20% (0,20,40,60,80,100)
                setProgress(workDataOf("progress" to i)) // Отправка прогресса
                delay(500) // Имитация работы
            }
            // Успех - передаем путь к сжатому файлу дальше
            return Result.success(workDataOf("image_path" to
"compressed_image.jpg"))
        } catch (e: Exception) {
            return Result.failure() // Ошибка - цепочка прервется
        }
    }
}

// 2. Воркер для добавления водяного знака

```

```

class WatermarkWorker(context: Context, params: WorkerParameters) :
CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        // Получаем путь от предыдущего воркера
        val inputPath = inputData.getString("image_path")
        try {
            // Имитация наложения водяного знака
            for (i in 0..100 step 25) { // Шаг 25% (0,25,50,75,100)
                setProgress(workDataOf("progress" to i))
                delay(600)
            }
            // Успех - передаем путь с водяным знаком
            return Result.success(workDataOf("image_path" to
"${inputPath}_watermarked.png"))
        } catch (e: Exception) {
            return Result.failure()
        }
    }
}

```

// 3. Воркер для загрузки в облако

```

class UploadWorker(context: Context, params: WorkerParameters) :
CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        // Получаем финальный путь от предыдущего воркера
        val finalPath = inputData.getString("image_path")
        try {
            // Имитация загрузки
            for (i in 0..100 step 10) { // Шаг 10% (более детальный прогресс)

```

```

        setProgress(workDataOf("progress" to i))
        delay(400)
    }
    // Успех - возвращаем URL загруженного файла
    return Result.success(workDataOf("final_url" to
"https://cloud.storage/$finalPath"))
    } catch (e: Exception) {
        return Result.failure()
    }
}
}

```

Интерфейс приложения (Compose UI)

```

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.runtime.livedata.observeAsState
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.work.*

@Composable
fun PhotoProcessingScreen() {
    val context = LocalContext.current
    val workManager = WorkManager.getInstance(context)
}

```

```

// ID последней цепочки задач
var workId by remember { mutableStateOf<java.util.UUID?>(null) }

// Наблюдение за состоянием всех воркеров с тегом "photo_chain"
val workInfos by
workManager.getWorkInfosByTagLiveData("photo_chain").observeAsState(emptyList())

// Поиск активного воркера
val activeWork = workInfos.find { it.state == WorkInfo.State.RUNNING ||
it.state == WorkInfo.State.ENQUEUED }
val isRunning = workInfos.any { it.state == WorkInfo.State.RUNNING ||
it.state == WorkInfo.State.ENQUEUED }
val isFinished = workInfos.all { it.state == WorkInfo.State.SUCCEEDED }
&& workInfos.isNotEmpty()
val isFailed = workInfos.any { it.state == WorkInfo.State.FAILED }

// Определение текущего этапа по тегам воркеров
val currentStatus = when {
    isRunning -> {
        val currentWorker = workInfos.find { it.state ==
WorkInfo.State.RUNNING }
        when (currentWorker?.tags?.firstOrNull { it.contains("Worker") }) {
            "CompressWorker" -> "Сжимаем фото..."
            "WatermarkWorker" -> "Добавляем водяной знак..."
            "UploadWorker" -> "Загружаем в облако..."
            else -> "Подготовка..."
        }
    }
}

```

```

    isFinished -> "Готово! Фото загружено"
    isFailed -> "Ошибка при обработке!"
    else -> "Ожидание запуска"
}

// Прогресс текущего воркера
val progress = activeWork?.progress?.getInt("progress", 0) ?: 0

// Результат от UploadWorker
val resultPath = workInfos.find { it.tags.contains("UploadWorker") }
    ?.outputData?.getString("final_url") ?: ""

Column(
    modifier = Modifier.fillMaxSize().padding(24.dp),
    verticalArrangement = Arrangement.Center,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    // Статус
    Text(text = currentStatus, fontSize = 20.sp, style =
MaterialTheme.typography.headlineMedium)

    Spacer(modifier = Modifier.height(16.dp))

    // Индикатор прогресса
    if (isRunning) {
        LinearProgressIndicator(
            progress = { progress / 100f },
            modifier = Modifier.fillMaxWidth(),
        )
    }
}

```

```

        Text("${progress}%")
    }

    // Результат
    if (isFinished) {
        Text("Путь: $resultPath", color = MaterialTheme.colorScheme.primary)
    }

    Spacer(modifier = Modifier.height(32.dp))

    // Кнопка запуска
    Button(
        onClick = {
            // Создание воркеров с тегами
            val compressReq =
OneTimeWorkRequestBuilder<CompressWorker>()
                .addTag("CompressWorker")
                .addTag("photo_chain")
                .build()

            val watermarkReq =
OneTimeWorkRequestBuilder<WatermarkWorker>()
                .addTag("WatermarkWorker")
                .addTag("photo_chain")
                .build()

            val uploadReq = OneTimeWorkRequestBuilder<UploadWorker>()
                .addTag("UploadWorker")
                .addTag("photo_chain")

```

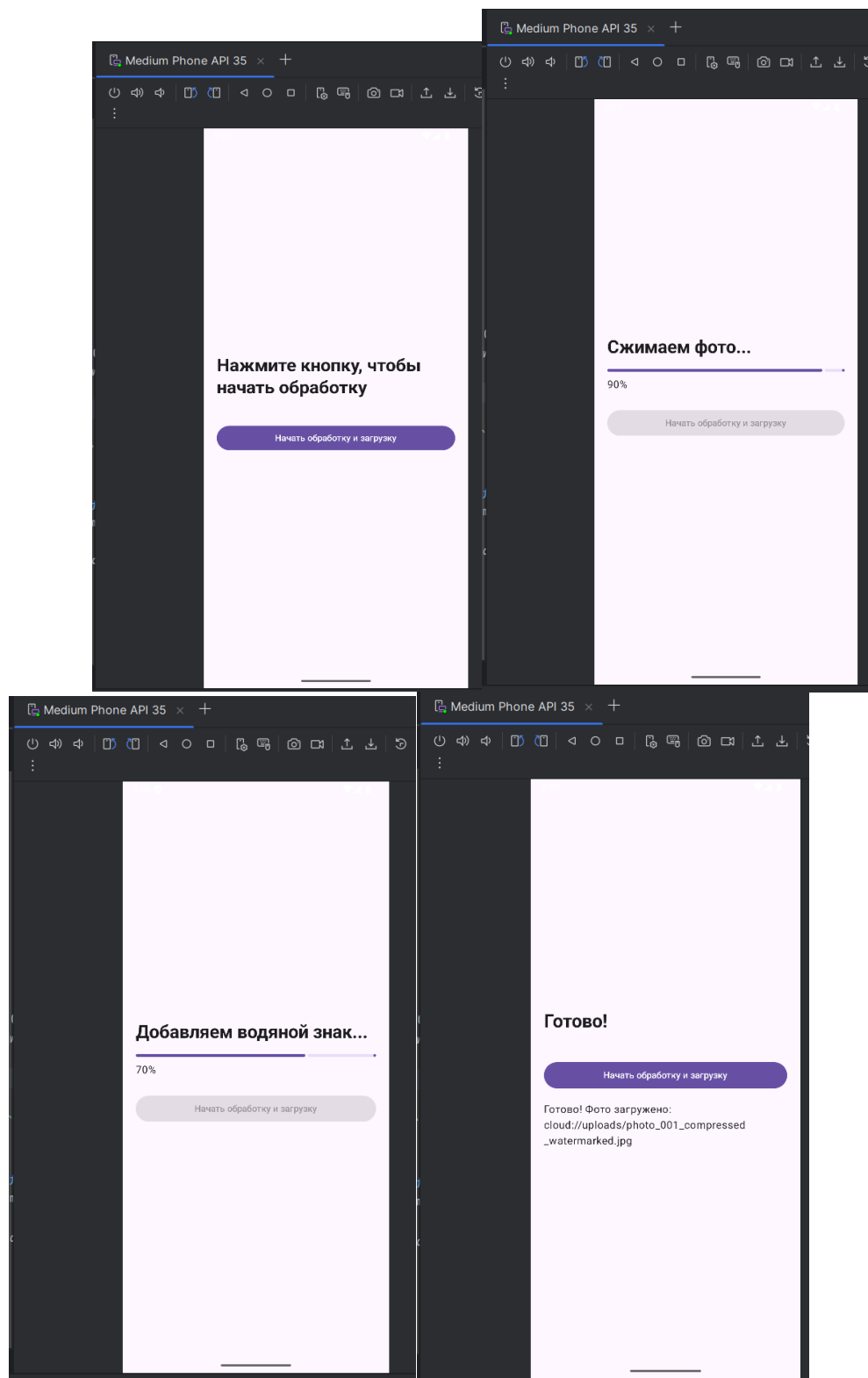


```

        .build()

        // Запуск уникальной цепочки (заменит предыдущую)
        workManager.beginUniqueWork(
            "photo_job",
            ExistingWorkPolicy.REPLACE, // Заменить если уже есть
            compressReq
        )
        .then(watermarkReq)
        .then(uploadReq)
        .enqueue()
    },
    enabled = !isRunning // Блокировка во время выполнения
) {
    Text("Начать обработку")
}
}
}

```



Задание 9 (Параллельная обработка): Загрузка нескольких файлов и финальная сборка итогового отчета о погоде

Функционал приложения:

Приложение собирает прогноз погоды для 3–4 городов одновременно (параллельно). Пользователь нажимает кнопку - начинается фоновая работа. Приложение показывает уведомление в статус-баре, которое обновляется по мере выполнения:

- «Загружаем погоду для 3 городов...»;
- «Готово: Москва и Лондон, Нью-Йорк в процессе...»;
- «Все данные получены, формируем отчёт...»;
- «Отчёт готов! Средняя температура +12°C».

Требования к реализации:

- Кнопка «Собрать прогноз»;
- После нажатия кнопки - сразу показывается **постоянное уведомление** (ongoing) с прогрессом;
- Уведомление обновляется каждые несколько секунд (или по мере завершения Worker'ов);
- Используется **Foreground Service** + WorkManager (чтобы уведомление могло жить долго);
- Или просто **WorkManager** с **setForegroundAsync** внутри Worker'ов (рекомендуемый способ с WorkManager 2.8+);
- Финальный Worker объединяет результаты и обновляет уведомление в последний раз;

1. Реализация Worker для загрузки погоды в городе

```
import android.content.Context
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import androidx.work.workDataOf
import kotlinx.coroutines.delay
import kotlin.random.Random

/**
 * Worker для загрузки погоды в фоне
```

```

* Получает город на вход, возвращает температуру
*/
class WeatherWorker(context: Context, params: WorkerParameters) :
CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        // Получаем входные данные (город)
        val city = inputData.getString("city") ?: "Unknown"

        // Имитация сетевого запроса
        // Случайная задержка от 2 до 5 секунд
        delay(Random.nextLong(2000, 5000))

        // Генерация случайной температуры от 10 до 25
        val temp = Random.nextInt(10, 25)

        // Возвращаем результат
        return Result.success(workDataOf(
            "city_result" to "$city: $temp°C", // Строка с результатом
            "temp" to temp // Числовое значение
        ))
    }
}

```

Финальный Worker для сборки отчета

```

import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.Context
import androidx.core.app.NotificationCompat

```

```

import androidx.work.CoroutineWorker
import androidx.work.ForegroundInfo
import androidx.work.WorkerParameters

/**
 * Worker для сбора результатов от нескольких WeatherWorker
 * Запускается в foreground режиме с уведомлением
 */
class ReportWorker(context: Context, params: WorkerParameters) :
    CoroutineWorker(context, params) {

    private val notificationManager =
context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager

    private val channelId = "weather_reports"

    override suspend fun doWork(): Result {
        // Создаем канал уведомлений (для Android 8+)
        createNotificationChannel()

        // Переводим воркер в foreground режим с уведомлением
        setForeground(createForegroundInfo("Формируем итоговый отчёт..."))

        // Получаем данные от всех WeatherWorker
        // Ищем строки с результатами (city: temp°C)
        val results = inputData.keyValueMap.values.filterIsInstance<Array<*>>()
            .flatMap { it.toList() }
            .filterIsInstance<String>()
    }
}

```

```

// Ищем числовые значения температур
val temps = inputData.keyValueMap.values.filterIsInstance<Int>()
val avgTemp = if (temps.isNotEmpty()) temps.average().toInt() else 0

val finalReport = "Отчёт готов! Средняя температура: $avgTemp°C"

// Обновляем уведомление финальным результатом
updateNotification(finalReport)

return Result.success()
}

// Создание foreground-уведомления с прогрессом
private fun createForegroundInfo(progress: String): ForegroundInfo {
    val notification = NotificationCompat.Builder(applicationContext,
channelId)
        .setContentTitle("Прогноз погоды")
        .setTicker("Загрузка...")
        .setContentText(progress)
        .setSmallIcon(android.R.drawable.ic_menu_day)
        .setOngoing(true) // Нельзя смахнуть
        .build()
    return ForegroundInfo(101, notification)
}

// Обновление уведомления (без ongoing)
private fun updateNotification(text: String) {
    val notification = NotificationCompat.Builder(applicationContext,
channelId)

```

```

        .setContentType("Прогноз погоды")
        .setContentText(text)
        .setSmallIcon(android.R.drawable.ic_menu_day)
        .build()
notificationManager.notify(101, notification)
}

// Создание канала уведомлений
private fun createNotificationChannel() {
    if (android.os.Build.VERSION.SDK_INT >=
android.os.Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            channelId,
            "Weather Report Channel",
            NotificationManager.IMPORTANCE_LOW // Без звука
        )
        notificationManager.createNotificationChannel(channel)
    }
}
}

```

Интерфейс и запуск цепочки (Compose UI)

```

@Composable
fun WeatherReportScreen() {
    val context = LocalContext.current
    val workManager = WorkManager.getInstance(context)

    // ID для отслеживания цепочки
    var workId by remember { mutableStateOf<WorkInfo?>(null) }
}

```

```

// Наблюдение за статусом последней работы
val workInfos by
workManager.getWorkInfosByTagLiveData("weather_chain").observeAsState(e
mptyList())

// Определение состояния
val isRunning = workInfos.any { it.state == WorkInfo.State.RUNNING ||
it.state == WorkInfo.State.ENQUEUED }
val isFinished = workInfos.any { it.state == WorkInfo.State.SUCCEEDED }

// Результат от ReportWorker
val reportResult = workInfos.find { it.tags.contains("ReportWorker") }
?.outputData?.getString("report") ?: ""

Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(24.dp),
    verticalArrangement = Arrangement.Center,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    // Кнопка запуска
    Button(
        onClick = {
            // Создаем задачи для разных городов
            val city1 = OneTimeWorkRequestBuilder<WeatherWorker>()
                .setInputData(workDataOf("city" to "Москва"))
                .addTag("WeatherWorker")
        }
    )
}

```



```

        .addTag("weather_chain")
        .build()

val city2 = OneTimeWorkRequestBuilder<WeatherWorker>()
    .setInputData(workDataOf("city" to "Лондон"))
    .addTag("WeatherWorker")
    .addTag("weather_chain")
    .build()

val city3 = OneTimeWorkRequestBuilder<WeatherWorker>()
    .setInputData(workDataOf("city" to "Нью-Йорк"))
    .addTag("WeatherWorker")
    .addTag("weather_chain")
    .build()

// ReportWorker собирает результаты
val reportWorker = OneTimeWorkRequestBuilder<ReportWorker>()
    .addTag("ReportWorker")
    .addTag("weather_chain")
    .build()

// Запускаем параллельно города, затем отчет
workManager.beginUniqueWork(
    "weather_job",
    ExistingWorkPolicy.REPLACE, // Заменить предыдущий
    listOf(city1, city2, city3)
)
    .then(reportWorker)
    .enqueue()

```

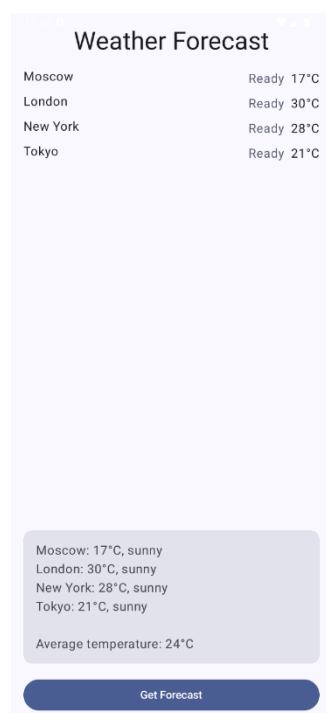
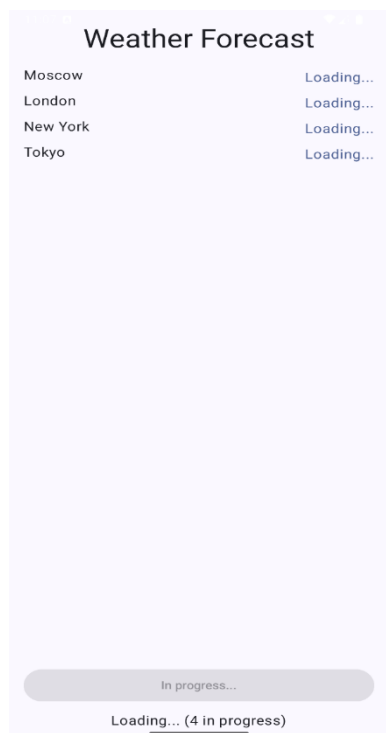
```

    },
    enabled = !isRunning // Блокировка во время выполнения
) {
    Text(if (isRunning) "Загрузка..." else "Собрать прогноз")
}

Spacer(modifier = Modifier.height(24.dp))

// Отображение результата
if (isFinished && reportResult.isNotEmpty()) {
    Text(
        text = reportResult,
        style = MaterialTheme.typography.headlineSmall
    )
}
}
}
}

```



Задание 10: Создание приложения для определения местоположения, а также сделать обратное геокодирование (получить адрес по координатам)

Функционал приложения:

Одна кнопка «Получить мой адрес». После нажатия (и предоставления разрешений):

- Показать индикатор загрузки;
- Получить текущие координаты (latitude, longitude);
- Преобразовать их в читаемый адрес (улица, город, страна и т.д.);
- Вывести адрес крупным текстом;
- Показать краткие координаты (lat / lng).

Требования к реализации:

- Использовать интерфейс: **FusedLocationProviderClient**;
- Обработать разрешения (**ACCESS_COARSE_LOCATION** + **ACCESS_FINE_LOCATION**);
- Можно использовать **Geocoder** для обратного геокодирования;
- Обработать ошибки (нет GPS, нет интернета, пользователь отказал в разрешении и т.д.).

2. Реализация логики (Compose + Google Play Services)

```
import android.Manifest
import android.annotation.SuppressLint
import android.content.Context
import android.location.Geocoder
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
```

```

import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.google.android.gms.location.LocationServices
import com.google.android.gms.location.Priority
import com.google.android.gms.tasks.CancellationTokenSource
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import java.util.Locale

@Composable
fun LocationScreen() {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    val locationClient = remember {
        LocationServices.getFusedLocationProviderClient(context) }

    // Состояния UI
    var addressText by remember { mutableStateOf("Нажмите кнопку для поиска") }
    var coordsText by remember { mutableStateOf("") }
    var isLoading by remember { mutableStateOf(false) }

    // Лаунчер для запроса разрешений на геолокацию
    val permissionLauncher = rememberLauncherForActivityResult(

```

```

        ActivityResultContracts.RequestMultiplePermissions()
    ) { permissions ->
        // Проверяем, дано ли разрешение FINE_LOCATION
        if (permissions[Manifest.permission.ACCESS_FINE_LOCATION] ==
true) {
            fetchLocation(context, locationClient, { isLoading = it }, { addr, coord ->
                addressText = addr
                coordsText = coord
            })
        } else {
            addressText = "Доступ к GPS отключен"
        }
    }

    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Индикатор загрузки
        if (isLoading) {
            CircularProgressIndicator()
            Spacer(modifier = Modifier.height(16.dp))
        }

        // Адрес
        Text(
            text = addressText,
            fontSize = 22.sp,

```

```

fontWeight = FontWeight.Bold,
modifier = Modifier.padding(bottom = 8.dp)
)

// Координаты
if (coordsText.isNotEmpty()) {
    Text(text = coordsText, color = MaterialTheme.colorScheme.secondary)
}

Spacer(modifier = Modifier.height(32.dp))

// Кнопка запроса геолокации
Button(
    onClick = {
        // Запрашиваем разрешения при нажатии
        permissionLauncher.launch(
            arrayOf(
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            )
        )
    },
    enabled = !isLoading
) {
    Text("Получить мой адрес")
}
}
}

```

```

/**
 * Получение текущего местоположения и преобразование в адрес
 */
@SuppressLint("MissingPermission")
private fun fetchLocation(
    context: Context,
    client: com.google.android.gms.location.FusedLocationProviderClient,
    setLoading: (Boolean) -> Unit,
    onResult: (String, String) -> Unit
) {
    setLoading(true)

    // Запрос текущего местоположения с высокой точностью
    client.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY,
CancellationTokenSource().token)
        .addOnSuccessListener { location ->
            if (location != null) {
                // Геокодирование в фоновом потоке (блокирующая операция)
                val geocoder = Geocoder(context, Locale.getDefault())

                // Используем корутину для переключения на IO поток
                (context as
androidx.activity.ComponentActivity).lifecycleScope.launch(Dispatchers.IO) {
                    try {
                        val addresses = geocoder.getFromLocation(location.latitude,
location.longitude, 1)
                        withContext(Dispatchers.Main) {
                            if (!addresses.isNullOrEmpty()) {
                                val addr = addresses[0]

```

```

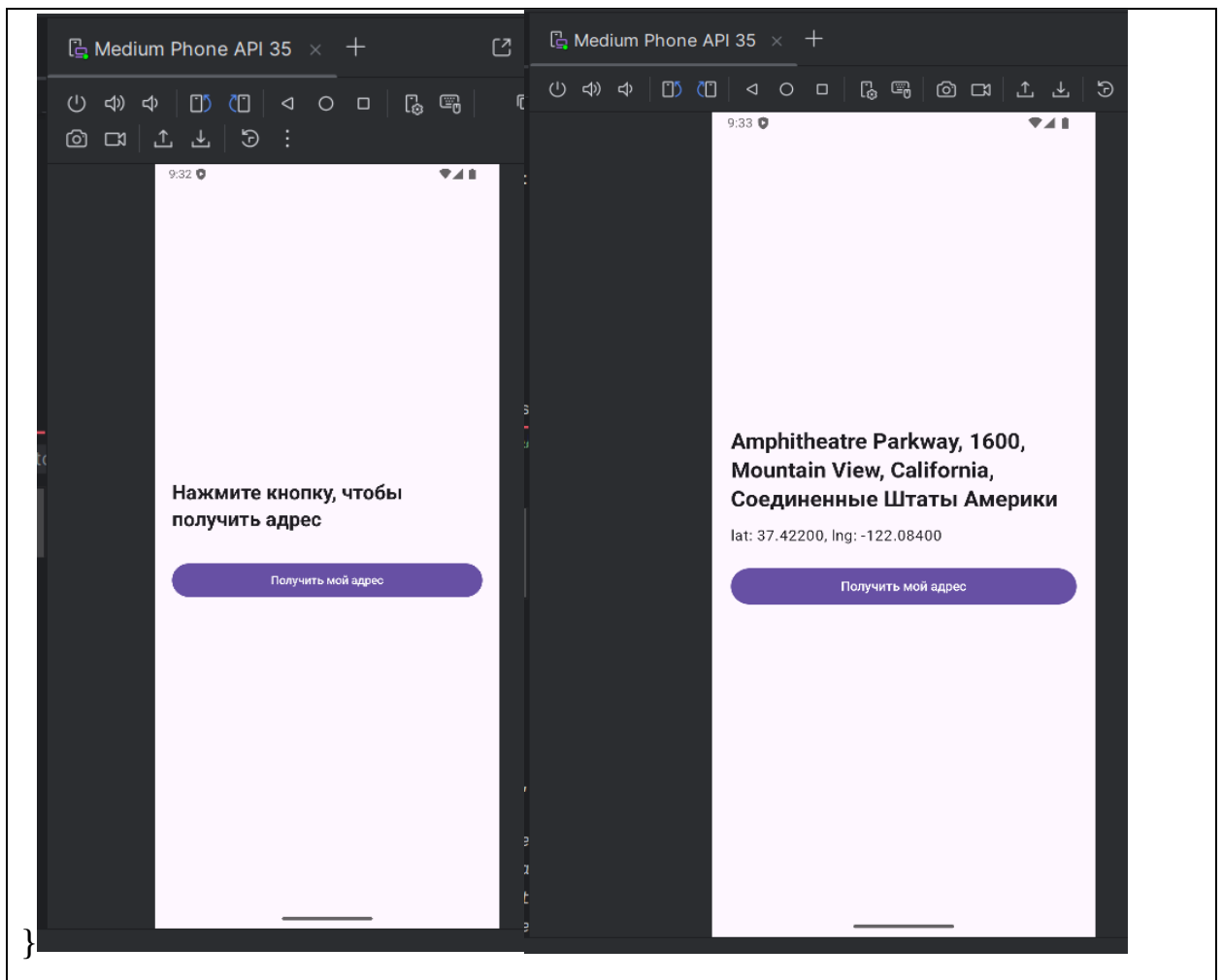
        // Формируем адрес: улица, город, страна
        val fullAddress = "${addr.thoroughfare ?: ""},
        ${addr.locality ?: ""}, ${addr.countryName ?: ""}"

        onResult(fullAddress, "Lat: ${location.latitude} / Lng:
        ${location.longitude}")

        } else {
            onResult("Адрес не найден", "")
        }
        setLoading(false)
    }
} catch (e: Exception) {
    withContext(Dispatchers.Main) {
        onResult("Ошибка геокодирования: ${e.message}", "")
        setLoading(false)
    }
}
} else {
    onResult("Не удалось получить координаты (включите GPS)", "")
    setLoading(false)
}
}

.addOnFailureListener { exception ->
    onResult("Ошибка GPS: ${exception.message}", "")
    setLoading(false)
}
}

```

Задание 11 (AlarmManager): Ежедневное напоминание о таблетке

Функционал приложения:

Пользователь хочет получать уведомление каждый день ровно в 20:00 с текстом «Время принять таблетку!». Приложение должно иметь следующий функционал:

- Позволять включить/выключить напоминание
- Показывать текущее состояние (включено / выключено)
- При нажатии кнопки «Включить» → ставит точный будильник на ближайшие 20:00 (сегодня или завтра)
- При нажатии «Выключить» → отменяет все будильники
- После перезагрузки телефона напоминание должно восстанавливаться автоматически

Требования к UI:

- Большой текст посередине: «Напоминание о таблетке» + текущее время следующего напоминания;
- Большая кнопка «Включить напоминание» / «Выключить напоминание»;
- Индикатор состояния: зелёный круг + «Включено» / серый + «Выключено»;
- После включения — показывается текст «Следующее напоминание: сегодня/завтра в 20:00»

Технические требования:

- Восстанавливать напоминание после перезагрузки устройства (BOOT_COMPLETED);
- Обработать разрешение POST_NOTIFICATIONS (Android 13+).

Настройка манифеста (AndroidManifest.xml)

```

<!-- Разрешения для уведомлений, точных будильников и автозапуска -->

<uses-permission
android:name="android.permission.POST_NOTIFICATIONS" /> <!-- Android
13+ -->

<uses-permission
android:name="android.permission.SCHEDULE_EXACT_ALARM" /> <!--
Точные будильники -->

<uses-permission
android:name="android.permission.RECEIVE_BOOT_COMPLETED" /> <!--
Автозапуск после перезагрузки -->

<application>

    <!-- Приемник для сработавшего будильника (не экспортирован) -->

    <receiver

        android:name=".AlarmReceiver"

        android:enabled="true"

        android:exported="false" /> <!-- Только для внутреннего использования
-->

    <!-- Приемник для события загрузки системы (экспортирован) -->

    <receiver

        android:name=".BootReceiver"

        android:enabled="true"

        android:exported="true"> <!-- Должен быть экспортирован для
системных событий -->

        <intent-filter>

```

```

        <action android:name="android.intent.action.BOOT_COMPLETED" />
<!-- Событие перезагрузки -->

    </intent-filter>

</receiver>

</application>

```

Реализация логики (Receiver и Будильник)

```

import android.app.*
import android.content.*
import android.os.Build
import androidx.core.app.NotificationCompat
import java.util.Calendar

/**
 * Приемник для сработавшего будильника
 */
class AlarmReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {

        val notificationManager =
context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager

        val channelId = "pill_reminders"

        // Создание канала для Android 8+

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {

            val channel = NotificationChannel(

```

```

        channelId,

        "Pill Reminders",

        NotificationManager.IMPORTANCE_HIGH // Важность высокая
(звук/вибрация)

    )

    notificationManager.createNotificationChannel(channel)
}

// Создание уведомления

val notification = NotificationCompat.Builder(context, channelId)

    .setSmallIcon(android.R.drawable.ic_lock_idle_alarm)

    .setContentTitle("Напоминание о таблетке")

    .setContentText("Время принять таблетку!")

    .setPriority(NotificationCompat.PRIORITY_HIGH)

    .setAutoCancel(true) // Исчезает при нажатии

    .build()

notificationManager.notify(1, notification)

// Автоматическое переназначение на завтра

AlarmHelper.setAlarm(context)
}
}

```

```

/**
 * Приемник для восстановления будильника после перезагрузки
 */
class BootReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {

        if (intent.action == Intent.ACTION_BOOT_COMPLETED) {

            val prefs = context.getSharedPreferences("pill_prefs",
Context.MODE_PRIVATE)

            // Проверяем, был ли будильник включен до перезагрузки

            if (prefs.getBoolean("alarm_enabled", false)) {

                AlarmHelper.setAlarm(context)

            }

        }

    }

}

/**
 * Helper для управления будильником
 */
object AlarmHelper {

    // Установка будильника на 20:00 каждый день

    fun setAlarm(context: Context) {

```

```

    val alarmManager =
context.getSystemService(Context.ALARM_SERVICE) as AlarmManager

    val intent = Intent(context, AlarmReceiver::class.java)

    val pendingIntent = PendingIntent.getBroadcast(

        context,

        0,

        intent,

        PendingIntent.FLAG_UPDATE_CURRENT or
PendingIntent.FLAG_IMMUTABLE

    )

// Установка времени на 20:00

val calendar = Calendar.getInstance().apply {

    set(Calendar.HOUR_OF_DAY, 20)

    set(Calendar.MINUTE, 0)

    set(Calendar.SECOND, 0)

    // Если время уже прошло сегодня, ставим на завтра
    if (before(Calendar.getInstance())) {

        add(Calendar.DATE, 1)

    }

}

// Проверка разрешения на точные будильники (Android 12+)

if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S &&
!alarmManager.canScheduleExactAlarms()) {

```

```
// Можно запросить разрешение у пользователя или использовать
неточный будильник
```

```
    return
}
```

```
// Точный будильник, который сработает даже в режиме
энергосбережения
```

```
alarmManager.setExactAndAllowWhileIdle(
    AlarmManager.RTC_WAKEUP, // Разбудит устройство
    calendar.timeInMillis,
    pendingIntent
)
}
```

```
// Отмена будильника
```

```
fun cancelAlarm(context: Context) {
    val alarmManager =
context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
    val intent = Intent(context, AlarmReceiver::class.java)
    val pendingIntent = PendingIntent.getBroadcast(
        context,
        0,
        intent,
        PendingIntent.FLAG_UPDATE_CURRENT or
        PendingIntent.FLAG_IMMUTABLE
    )
    alarmManager.cancel(pendingIntent)
}
```



```
)  
    alarmManager.cancel(pendingIntent)  
}  
}
```

Интерфейс (Compose UI)

```
import androidx.compose.foundation.background  
import androidx.compose.foundation.layout.*  
import androidx.compose.foundation.shape.CircleShape  
import androidx.compose.material3.*  
import androidx.compose.runtime.*  
import androidx.compose.ui.Alignment  
import androidx.compose.ui.Modifier  
import androidx.compose.ui.graphics.Color  
import androidx.compose.ui.platform.LocalContext  
import androidx.compose.ui.unit.dp  
import androidx.compose.ui.unit.sp  
import android.Manifest  
import android.os.Build  
import androidx.activity.compose.rememberLauncherForActivityResult  
import androidx.activity.result.contract.ActivityResultContracts  
  
@Composable  
fun PillReminderScreen() {
```

```

val context = LocalContext.current

// SharedPreferences для сохранения состояния будильника

val prefs = remember { context.getSharedPreferences("pill_prefs",
Context.MODE_PRIVATE) }

var isEnabled by remember {
mutableStateOf(prefs.getBoolean("alarm_enabled", false)) }


// Лаунчер для запроса разрешения на уведомления (Android 13+)

val permissionLauncher = rememberLauncherForActivityResult(

    ActivityResultContracts.RequestPermission()

) { granted ->

    // Можно показать сообщение если отказано

}


Column(

    modifier = Modifier.fillMaxSize().padding(32.dp),

    horizontalAlignment = Alignment.CenterHorizontally,

    verticalArrangement = Arrangement.Center

) {

    // Заголовок

    Text("Напоминание о таблетке", fontSize = 24.sp)


    Spacer(modifier = Modifier.height(16.dp))

```

```

// Статус (цветной индикатор + текст)
Row(verticalAlignment = Alignment.CenterVertically) {
    Box(
        modifier = Modifier
            .size(16.dp)
            .background(if (isEnabled) Color.Green else Color.Gray,
CircleShape)
    )
    Spacer(modifier = Modifier.width(8.dp))
    Text(if (isEnabled) "Включено" else "Выключено")
}

Spacer(modifier = Modifier.height(8.dp))

// Информация о времени следующего напоминания
if (isEnabled) {
    Text("Следующее напоминание: сегодня/завтра в 20:00", color =
Color.Gray)
}

Spacer(modifier = Modifier.height(32.dp))

// Кнопка включения/выключения
Button(

```

```

onClick = {

    // Запрос разрешения на уведомления для Android 13+

    if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {

permissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)

    }

    // Переключение будильника

    if (isEnabled) {

        AlarmHelper.cancelAlarm(context)

        prefs.edit().putBoolean("alarm_enabled", false).apply()

    } else {

        AlarmHelper.setAlarm(context)

        prefs.edit().putBoolean("alarm_enabled", true).apply()

    }

    isEnabled = !isEnabled

},

modifier = Modifier

    .fillMaxWidth()

    .height(56.dp),

colors = ButtonDefaults.buttonColors(

    containerColor = if (isEnabled) Color.LightGray else
MaterialTheme.colorScheme.primary

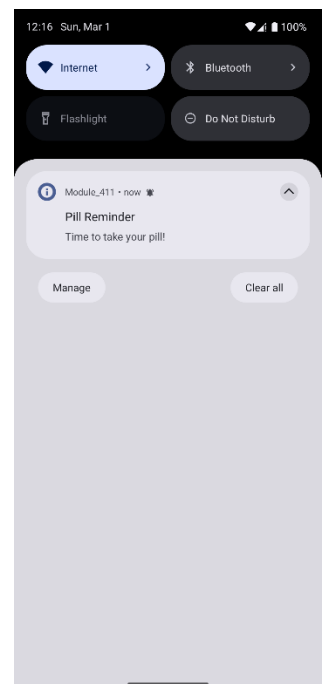
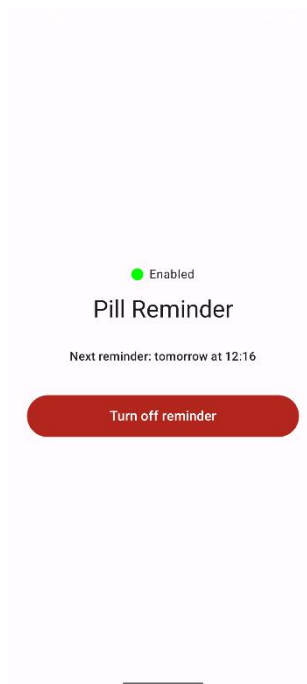
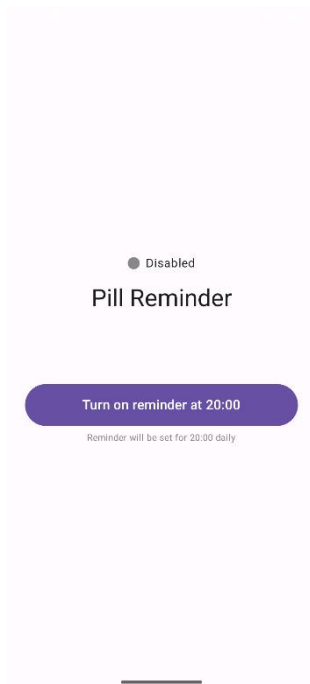
)

```

```

    ) {
        Text(if (isEnabled) "Выключить напоминание" else "Включить
напоминание")
    }
}
}
}

```



Задание 12 (cold Flow): Генератор случайных фактов о животных

Функционал приложения:

На экране кнопка «Новый факт!». При нажатии генерируется новый случайный факт об животном (из списка или с имитацией задержки). Набор фактов можно взять отсюда: <https://dzen.ru/a/ZKA2BQgB8EUGYNjw> Факт отображается в красивом Card с анимацией появления. Каждый новый collect запускает генерацию заново (cold Flow).

Требования:

- ViewModel возвращает функцию getRandomFact(): Flow<String>;
- Внутри Flow — delay(1500–3000 мс) для имитации загрузки + emit факта;
- UI: кнопка + Card с фактом + индикатор загрузки;
- При повороте экрана факт не генерируется заново (только при нажатии кнопки);
- Минимум 10–15 фактов в списке.

Примерный UI

Реализация ViewModel

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.flow.onStart
import kotlinx.coroutines.launch
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.State
```

```

import kotlin.random.Random

/**
 * ViewModel для загрузки случайных фактов о животных
 */
class AnimalFactsViewModel : ViewModel() {

    // База фактов
    private val facts = listOf(
        "Сердце кита бьется всего 9 раз в минуту.",
        "Слоны — единственные животные, которые не умеют прыгать.",
        "У кошек по 32 мышцы в каждом ухе.",
        "Отпечатки пальцев коал практически неотличимы от человеческих.",
        "Морские коньки — единственные существа, у которых рожают
самцы.",
        "У осьминога три сердца и голубая кровь.",
        "Медведи гризли могут бегать со скоростью до 48 км/ч.",
        "Муравьи никогда не спят и у них нет легких.",
        "Язык синего кита весит столько же, сколько целый слон.",
        "Коровы могут спать стоя, но видят сны только когда лежат.",
        "Улитка может спать три года подряд.",
        "Тигры имеют полосатую кожу, а не только мех.",
        "Крокодилы глотают камни, чтобы глубже нырять.",
        "Белки сажают тысячи деревьев каждый год, забывая, куда спрятали
орехи."
    )

    // Состояния UI
    private val _currentFact = mutableStateOf<String?>(null)

```

```

val currentFact: State<String?> = _currentFact // Текущий факт

private val _isLoading = mutableStateOf(false)
val isLoading: State<Boolean> = _isLoading // Флаг загрузки

/**
 * Cold Flow: создает поток данных
 * Каждый collect() запускает новую эмиссию
 */
fun getRandomFactFlow(): Flow<String> = flow {
    val randomFact = facts[Random.nextInt(facts.size)]
    delay(2000) // Имитация долгой загрузки (2 секунды)
    emit(randomFact) // Отправка результата
}

/**
 * Загрузка нового случайного факта
 */
fun loadNewFact() {
    viewModelScope.launch { // Запуск в корутине ViewModel
        getRandomFactFlow()
            .onStart { _isLoading.value = true } // Сразу включаем загрузку
            .collect { fact ->
                _currentFact.value = fact // Сохраняем факт
                _isLoading.value = false // Выключаем загрузку
            }
    }
}
}

```


Интерфейс приложения (Compose UI)

```
import androidx.compose.animation.*
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
fun AnimalFactsScreen(vm: AnimalFactsViewModel = viewModel()) {
    // Получение состояний из ViewModel
    val fact by vm.currentFact
    val isLoading by vm.isLoading

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Контейнер для контента с фиксированной высотой
        Box(
            modifier = Modifier
                .height(200.dp) // Фиксированная высота для стабильности
        )
    }
}
```

```

        .fillMaxWidth(),
        contentAlignment = Alignment.Center
    ) {
        if (isLoading) {
            // Индикатор загрузки
            CircularProgressIndicator()
        } else {
            // Анимированное появление карточки с фактом
            AnimatedVisibility(
                visible = fact != null, // Показываем только если есть факт
                enter = fadeIn() + expandVertically(), // Анимация появления
                exit = fadeOut() // Анимация исчезновения
            ) {
                Card(
                    elevation = CardDefaults.cardElevation(defaultElevation = 8.dp),
                    colors = CardDefaults.cardColors(
                        containerColor =
MaterialTheme.colorScheme.primaryContainer
                    ),
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text(
                        text = fact ?: "",
                        modifier = Modifier.padding(20.dp),
                        fontSize = 18.sp,
                        textAlign = TextAlign.Center,
                        style = MaterialTheme.typography.bodyLarge
                    )
                }
            }
        }
    }

```

```

    }

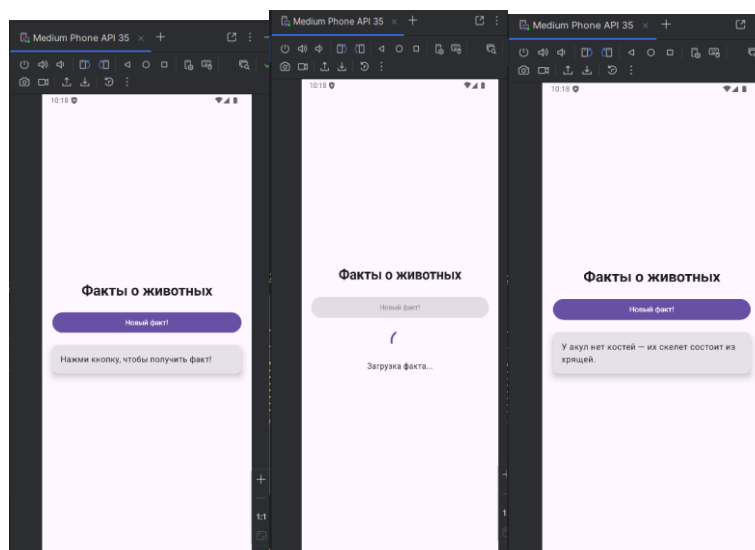
    }

}

Spacer(modifier = Modifier.height(32.dp))

// Кнопка загрузки факта
Button(
    onClick = { vm.loadNewFact() },
    enabled = !isLoading, // Блокировка во время загрузки
    modifier = Modifier
        .height(56.dp)
        .fillMaxWidth(0.7f) // 70% ширины экрана
) {
    Text(if (fact == null) "Узнать факт!" else "Новый факт!")
}
}
}

```



Задание 13 (StateFlow): Живой курс валют

Функционал приложения:

Экран показывает текущий курс доллара к рублю. Курс обновляется каждые 5 секунд (имитация). Есть кнопка «Обновить сейчас» — принудительно запрашивает новый курс.

Требования:

- ViewModel с MutableStateFlow<Double> (курс);
- В init — запускается корутина, которая каждые 5 секунд эммитит случайный курс (например, 90.5 ± 2.0);
- При нажатии кнопки — принудительно генерируется новый курс;
- UI: большой текст с курсом + стрелка вверх/вниз (зелёная/красная) при изменении + кнопка «Обновить»;
- При повороте экрана курс остаётся тем же (StateFlow сохраняет значение).

Примерный UI

Реализация ViewModel

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch
import kotlin.random.Random
```

```

/**
 * ViewModel для отслеживания изменения курса валют
 * Автоматически обновляет курс каждые 5 секунд
 */
class CurrencyViewModel : ViewModel() {

    // Текущий курс (изменяемый поток)
    private val _rate = MutableStateFlow(92.0) // Начальный курс
    val rate: StateFlow<Double> = _rate.asStateFlow() // Публичный
    неизменяемый поток

    // Предыдущий курс для сравнения
    private val _previousRate = MutableStateFlow(92.0)
    val previousRate: StateFlow<Double> = _previousRate.asStateFlow()

    init {
        // Запуск автоматического обновления при создании ViewModel
        startCurrencyUpdates()
    }

    /**
     * Запуск бесконечного цикла обновления курса
     */
    private fun startCurrencyUpdates() {
        viewModelScope.launch {
            while (isActive) { // Пока ViewModel активна
                delay(5000) // Пауза 5 секунд между обновлениями
                updateRate()
            }
        }
    }
}

```

```

    }
}

/**
 * Обновление курса случайным значением
 * Сохраняет предыдущее значение для сравнения
 */
fun updateRate() {
    // Сохраняем текущий курс как предыдущий
    _previousRate.value = _rate.value

    // Генерируем новый курс: 88.5 + случайное число от 0 до 4
    // Диапазон: 88.5 - 92.5
    val nextRate = 88.5 + Random.nextDouble() * 4.0

    // Форматируем до 2 знаков после запятой
    _rate.value = String.format("%.2f", nextRate)
        .replace(",", ".") // Замена запятой на точку (для локалей)
        .toDouble()
}
}

```

Интерфейс приложения (Compose UI)

```

import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowDropDown
import androidx.compose.material.icons.filled.KeyboardArrowUp
import androidx.compose.material3.*
import androidx.compose.runtime.*

```

```

import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
fun CurrencyScreen(vm: CurrencyViewModel = viewModel()) {
    // Подписываемся на StateFlow из ViewModel
    // collectAsState() преобразует Flow в Compose State для автоматического
    обновления UI

    val currentRate by vm.rate.collectAsState()
    val oldRate by vm.previousRate.collectAsState()

    // Определяем направление изменения курса
    val isIncreasing = currentRate >= oldRate
    // Цвет в зависимости от роста/падения (зеленый/красный)
    val statusColor = if (isIncreasing) Color(0xFF4CAF50) else
    Color(0xFFFF44336)

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {

```

```

// Заголовок
Text(text = "Курс USD/RUB", fontSize = 18.sp, color = Color.Gray)

Spacer(modifier = Modifier.height(8.dp))

// Строка с курсом и иконкой направления
Row(verticalAlignment = Alignment.CenterVertically) {
    Text(
        text = "$currentRate ₽",
        fontSize = 48.sp,
        fontWeight = FontWeight.Bold
    )

    // Иконка: стрелка вверх при росте, вниз при падении
    Icon(
        imageVector = if (isIncreasing) Icons.Default.KeyboardArrowUp else
Icons.Default.ArrowDropDown,
        contentDescription = null,
        tint = statusColor,
        modifier = Modifier.size(48.dp)
    )
}

// Текстовый статус
Text(
    text = if (isIncreasing) "Курс растет" else "Курс падает",
    color = statusColor,
    fontWeight = FontWeight.Medium
)

```



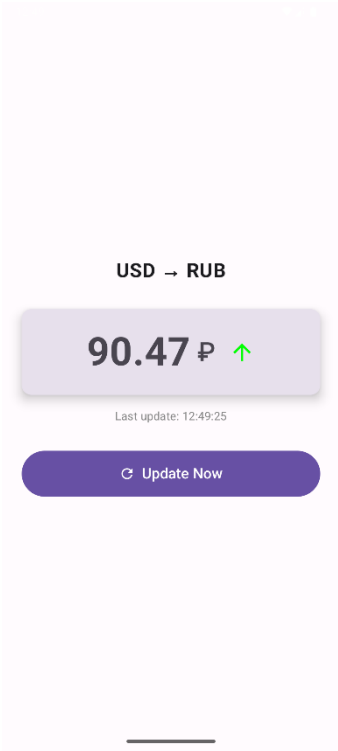
```

Spacer(modifier = Modifier.height(48.dp))

// Кнопка ручного обновления
Button(
    onClick = { vm.updateRate() },
    modifier = Modifier
        .fillMaxWidth(0.6f)
        .height(50.dp)
) {
    Text("Обновить сейчас")
}

// Информация об автообновлении
Text(
    text = "Автообновление каждые 5 сек",
    modifier = Modifier.padding(top = 16.dp),
    fontSize = 12.sp,
    color = Color.LightGray
)
}
}

```



Задание 14: Компас

Функционал приложения:

Приложение показывает направление на магнитный север с помощью встроенных сенсоров смартфона. Основные возможности:

1. Отображение азимута

- Показывает текущий азимут (угол от магнитного севера) в градусах (0° — север, 90° — восток, 180° — юг, 270° — запад);
- Значение обновляется в реальном времени при повороте устройства.

2. Анимированная стрелка компаса

- Стрелка плавно поворачивается в сторону севера;
- Красная часть стрелки указывает на север, серая — на юг;
- Поворот анимирован (без резких скачков).

3. Автоматическое управление сенсорами

- Слушатель включается, когда экран виден (onResume);
- Слушатель выключается, когда экран не виден (onPause) — экономия батареи;
- При отсутствии нужного сенсора — показывается сообщение об ошибке.

4. Сохранение поведения при повороте экрана

- Стрелка и значения не «сбрасываются» при изменении ориентации устройства.

Требования к UI:

UI должен быть минималистичным, стильным и удобным для использования в тёмное время суток.

1. Центральный элемент — компас

- Круглый диск диаметром $\sim 70\text{--}80\%$ ширины экрана;
- Стрелка компаса (двухцветная: красная на север, чёрная на юг);

- Стрелка занимает ~80% диаметра круга;
- Над стрелкой — крупная буква «N»;

2. Текстовая информация

- Вверху экрана — заголовок «Компас»
- Под стрелкой — крупный текст азимута: «Азимут: 142°»;

3. Сообщение об ошибке (если сенсор отсутствует)

- Большой красный текст посередине: «Устройство не поддерживает датчик ориентации».

Реализация логики сенсоров (SensorHandler)

```
import android.content.Context
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import androidx.compose.runtime.*

/**
 * Менеджер для работы с компасом через датчики устройства
 * Использует акселерометр и магнитометр для определения азимута
 */
class CompassManager(context: Context) : SensorEventListener {

    private val sensorManager =
context.getSystemService(Context.SENSOR_SERVICE) as SensorManager
    private val accelerometer =
sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER) //
Акселерометр
    private val magnetometer =
sensorManager.getDefaultSensor(Sensor.TYPE_MAGNETIC_FIELD) //
```

Магнитометр

```
// Массивы для хранения показаний датчиков
private var gravity = FloatArray(3)    // Данные акселерометра
private var geomagnetic = FloatArray(3) // Данные магнитометра

// Состояния для Compose (наблюдаемые значения)
var azimuth by mutableStateOf(0f)      // Текущий азимут в градусах
var isSupported by mutableStateOf(accelerometer != null && magnetometer
!= null) // Проверка наличия датчиков

/**
 * Запуск отслеживания датчиков
 */
fun start() {
    sensorManager.registerListener(this, accelerometer,
SensorManager.SENSOR_DELAY_UI)
    sensorManager.registerListener(this, magnetometer,
SensorManager.SENSOR_DELAY_UI)
}

/**
 * Остановка отслеживания датчиков
 */
fun stop() {
    sensorManager.unregisterListener(this)
}

/**
```

```

* Вызывается при изменении показаний датчиков
*/
override fun onSensorChanged(event: SensorEvent) {
    // Сохраняем показания соответствующих датчиков
    when (event.sensor.type) {
        Sensor.TYPE_ACCELEROMETER -> gravity = event.values
        Sensor.TYPE_MAGNETIC_FIELD -> geomagnetic = event.values
    }

    // Вычисляем матрицу поворота и ориентацию
    val rotationMatrix = FloatArray(9)
    val inclinationMatrix = FloatArray(9)

    if (SensorManager.getRotationMatrix(rotationMatrix, inclinationMatrix,
gravity, geomagnetic)) {
        val orientation = FloatArray(3)
        SensorManager.getOrientation(rotationMatrix, orientation)

        // Преобразуем радианы в градусы
        // Инвертируем для правильного отображения (по часовой стрелке)
        azimuth = -Math.toDegrees(orientation[0].toDouble()).toFloat()

        // Нормализуем в диапазон 0-360
        if (azimuth < 0) azimuth += 360f
    }
}

override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
    // Не используется, но требуется интерфейсом

```

```
}  
  
}
```

Интерфейс приложения (Compose UI)

```
import androidx.compose.animation.core.animateFloatAsState  
import androidx.compose.animation.core.spring  
import androidx.compose.foundation.Canvas  
import androidx.compose.foundation.background  
import androidx.compose.foundation.layout.*  
import androidx.compose.material3.MaterialTheme  
import androidx.compose.material3.Text  
import androidx.compose.runtime.*  
import androidx.compose.ui.Alignment  
import androidx.compose.ui.Modifier  
import androidx.compose.ui.geometry.Offset  
import androidx.compose.ui.graphics.Color  
import androidx.compose.ui.graphics.Path  
import androidx.compose.ui.graphics.drawscope.rotate  
import androidx.compose.ui.platform.LocalContext  
import androidx.compose.ui.platform.LocalLifecycleOwner  
import androidx.compose.ui.text.font.FontWeight  
import androidx.compose.ui.unit.dp  
import androidx.compose.ui.unit.sp  
import androidx.lifecycle.Lifecycle  
import androidx.lifecycle.LifecycleEventObserver  
  
@Composable  
fun CompassScreen() {  
    val context = LocalContext.current
```

```

val compassManager = remember { CompassManager(context) }
val lifecycleOwner = LocalLifecycleOwner.current

// Управление жизненным циклом сенсоров
DisposableEffect(lifecycleOwner) {
    val observer = LifecycleEventObserver { _, event ->
        if (event == Lifecycle.Event.ON_RESUME) compassManager.start()
        else if (event == Lifecycle.Event.ON_PAUSE) compassManager.stop()
    }
    lifecycleOwner.lifecycle.addObserver(observer)
    onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
}

// Плавная анимация поворота (инвертируем азимут для указания на север)
val animatedAzimuth by animateFloatAsState(
    targetValue = -compassManager.azimuth,
    animationSpec = spring(stiffness = 50f),
    label = "CompassRotation"
)

Column(
    modifier =
Modifier.fillMaxSize().background(Color(0xFF121212)).padding(16.dp),
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Компас", color = Color.White, fontSize = 24.sp, fontWeight =
FontWeight.Bold)
}

```



```

        Spacer(modifier = Modifier.weight(1f))

        if (!compassManager.isSupported) {
            Text("Устройство не поддерживает датчик ориентации", color =
Color.Red, textAlign = androidx.compose.ui.text.style.TextAlign.Center)
        } else {
            CompassView(animatedAzimuth)

            Spacer(modifier = Modifier.height(32.dp))

            val displayAzimuth = ((compassManager.azimuth.toInt() + 360) % 360)
            Text("Азимут: $displayAzimuth°", color = Color.White, fontSize =
28.sp)
        }

        Spacer(modifier = Modifier.weight(1f))
    }
}

@Composable
fun CompassView(rotation: Float) {
    Box(contentAlignment = Alignment.Center, modifier =
Modifier.fillMaxWidth(0.8f).aspectRatio(1f)) {
        Canvas(modifier = Modifier.fillMaxSize()) {
            val center = Offset(size.width / 2, size.height / 2)
            val radius = size.width / 2

            // Рисуем внешний круг (циферблат)
            drawCircle(color = Color.DarkGray, radius = radius, style =

```

```
androidx.compose.ui.graphics.drawscope.Stroke(4f))
```

```
rotate(rotation, pivot = center) {
```

```
    // Буква N
```

```
    // Рисуем стрелку
```

```
    val arrowWidth = 40f
```

```
    val arrowHeight = radius * 0.8f
```

```
    // Красная половина (Север)
```

```
    val northPath = Path().apply {
```

```
        moveTo(center.x, center.y - arrowHeight)
```

```
       .lineTo(center.x - arrowWidth, center.y)
```

```
       .lineTo(center.x + arrowWidth, center.y)
```

```
        close()
```

```
    }
```

```
    drawPath(northPath, Color.Red)
```

```
    // Серая половина (Юг)
```

```
    val southPath = Path().apply {
```

```
        moveTo(center.x, center.y + arrowHeight)
```

```
       .lineTo(center.x - arrowWidth, center.y)
```

```
       .lineTo(center.x + arrowWidth, center.y)
```

```
        close()
```

```
    }
```

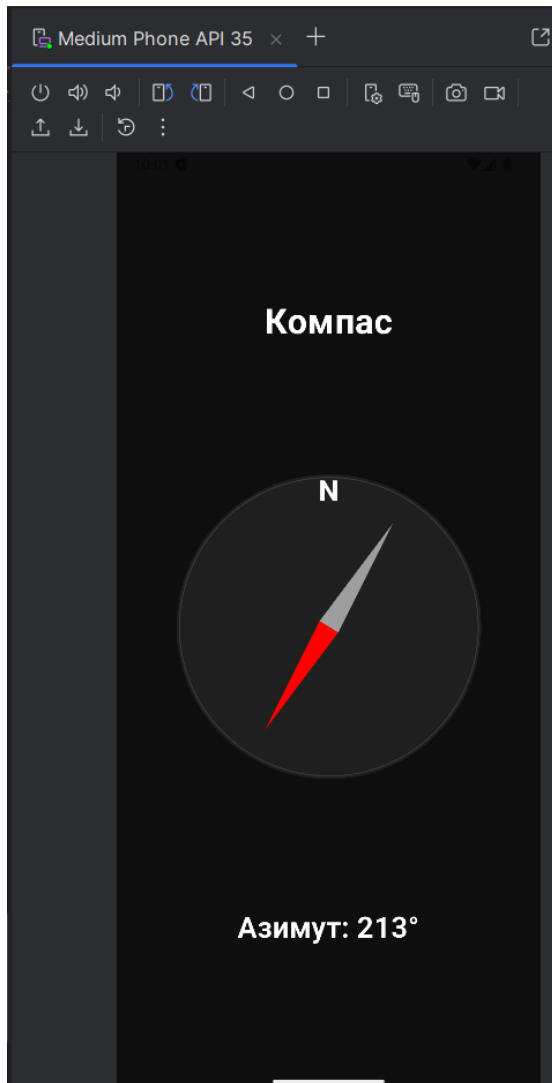
```
    drawPath(southPath, Color.Gray)
```

```
}
```

```
}
```

```
    // Буква N за пределами вращающегося холста для стабильности или  
    внутри вращения
```

```
Text("N", color = Color.Red, fontSize = 32.sp, fontWeight =  
FontWeight.Black,  
    modifier = Modifier.align(Alignment.TopCenter).offset(y = (-10).dp))  
}  
}
```



Выводы:

В ходе выполнения практической работы были изучены и применены на практике ключевые концепции разработки Android-приложений. Рассмотрены механизмы фоновой обработки задач с использованием WorkManager, работа с AlarmManager и BroadcastReceiver для создания точных уведомлений, а также интеграция датчиков устройства. Полученные навыки позволили создать приложения с современным интерфейсом на Jetpack Compose, корректной обработкой разрешений и оптимизированным потреблением ресурсов устройства.